



机器狗手掌跟踪

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程简述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (3学时)
- 六、模型应用及其部署 (4学时)
- 七、运行示例

目录

一、实验前置准备 (0.5学时)

1.1 实验描述

1.2 实验目的

1.3 实验建议

1.4 实验环境

1.5 背景知识

1.6 实验道具讲解

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (3学时)

六、模型应用及其部署 (4学时)

七、运行示例

1.1 实验描述

本实验的目标是基于cvzone计算机视觉库实现机器狗的目标追踪如手掌跟踪，识别手掌的关键点，例如手指尖和关节。通过这些关键点计算出实验中所需要的距离和角度来进行进一步的操作。在实验过程中，我们在对机器狗的精准控制之上，对手掌关键点进行解算，通过得到的数据来调整机器狗的动作。

目录

一、实验前置准备 (0.5学时)

1.1 实验描述

1.2 实验目的

1.3 实验建议

1.4 实验环境

1.5 背景知识

1.6 实验道具讲解

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (3学时)

六、模型应用及其部署 (4学时)

七、运行示例

1.2 实验目的

- 学生将学习如何捕获实时视频流，并使用计算机视觉库进行图像处理和分析
- 学习如何使用cvzone库和MediaPipe Hands模型进行手掌检测与关键点提取
- 理解手掌关键点的含义及其在计算机视觉中的应用
- 学生将了解如何根据手掌关键点计算手掌的距离和角度

目录

一、实验前置准备 (0.5学时)

1.1 实验描述

1.2 实验目的

1.3 实验建议

1.4 实验环境

1.5 背景知识

1.6 实验道具讲解

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (3学时)

六、模型应用及其部署 (4学时)

七、运行示例

1.3 实验建议

本实验主要是在YOLOv5上用Python编程，模型转换过程可以在自己的PC上进行，程序执行则需要在华为昇腾Atlas 200I DK开发板（Linux）上执行。实验中未注明的执行地方的，一律在开发板上执行。

Python编程

Quis ut consequat dignissim dolore
volutpat ut placerat.

OpenCV库

In amet sit dolor ea sanctus at duo
sadipscinc voluptua.

计算机视觉基础

Vel dolor stet wisi dolore magna et rebum
nam vero ea.

数据挖掘

At gubergren dolor ipsum suscipit diam
vulputate justo no.



目录

一、实验前置准备 (0.5学时)

1.1 实验描述

1.2 实验目的

1.3 实验建议

1.4 实验环境

1.5 背景知识

1.6 实验道具讲解

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (3学时)

六、模型应用及其部署 (4学时)

七、运行示例

1.4实验环境——实验硬件设备

本实验需要的准备的硬件如下：（详见章节3.1、3.4）

1



机器狗

2



PC

3



昇腾开发板

4



读卡器

5



摄像头

1.4实验环境——实验软件环境

本实验需要的准备的软件环境如下：（详见章节3.2、3.3）

1



MobaXterm

2



Pycharm

3


CV Zone

CV Zone

4



Anaconda

目录

一、实验前置准备 (0.5学时)

1.1 实验描述

1.2 实验目的

1.3 实验建议

1.4 实验环境

1.5 背景知识

1.6 实验道具讲解

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (3学时)

六、模型应用及其部署 (4学时)

七、运行示例

1.5 背景知识

(1) OpenCV (Open Source Computer Vision Library)



是一个开源的计算机视觉和机器学习软件库。它由Intel公司在1999年首次发布，现在由OpenCV基金会维护。

(2) CVZone



一个由 Adrian Rosebrock 创建的开源库，专门用于计算机视觉任务，特别是那些使用 Python 和 OpenCV 的任务。

(3) MediaPipe



一个由Google开发的开源跨平台框架，用于构建多模态应用，如计算机视觉、音频、传感器数据处理等。它特别适用于实时多媒体应用的开发，提供了一系列的工具和预构建的模块，使得开发者能够快速实现复杂的机器学习管道。

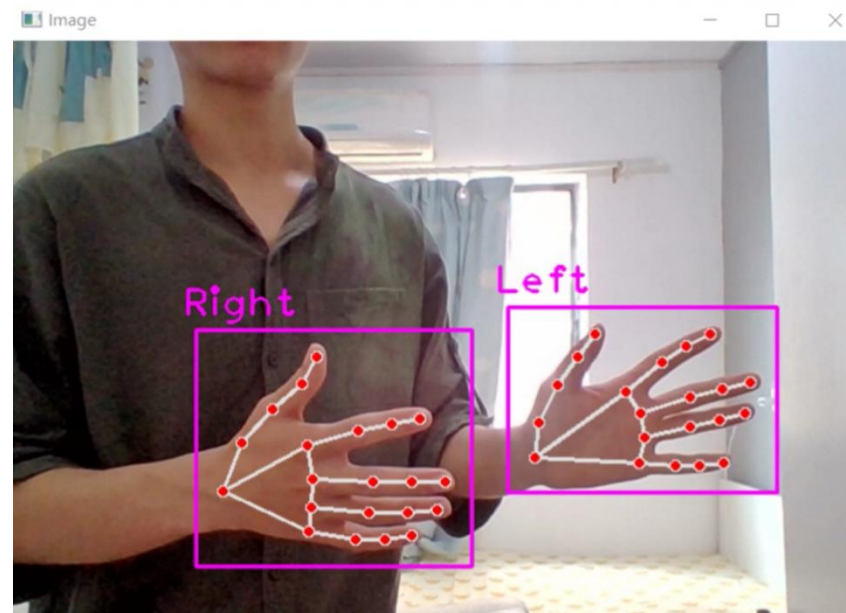
1.5 背景知识

(4) 手掌特征检测

是计算机视觉和人机交互领域的一个重要研究方向。这项技术能够使计算机识别和理解手掌的位置、姿势和运动，进而实现对机器人、虚拟角色或其他交互系统的控制。

(5) 特征点姿态解算 (Feature Point Pose Estimation)

特征点姿态解算是计算机视觉和机器人学中的一个关键技术，它涉及到确定图像中特征点的位置以及它们在三维空间中的姿态（位置和方向）。这项技术在多个领域有着广泛的应用，包括机器人导航、增强现实、三维重建、动作识别等



目录

一、实验前置准备 (0.5学时)

1.1 实验描述

1.2 实验目的

1.3 实验建议

1.4 实验环境

1.5 背景知识

1.6 实验道具讲解

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (3学时)

六、模型应用及其部署 (4学时)

七、运行示例

1.6实验道具讲解

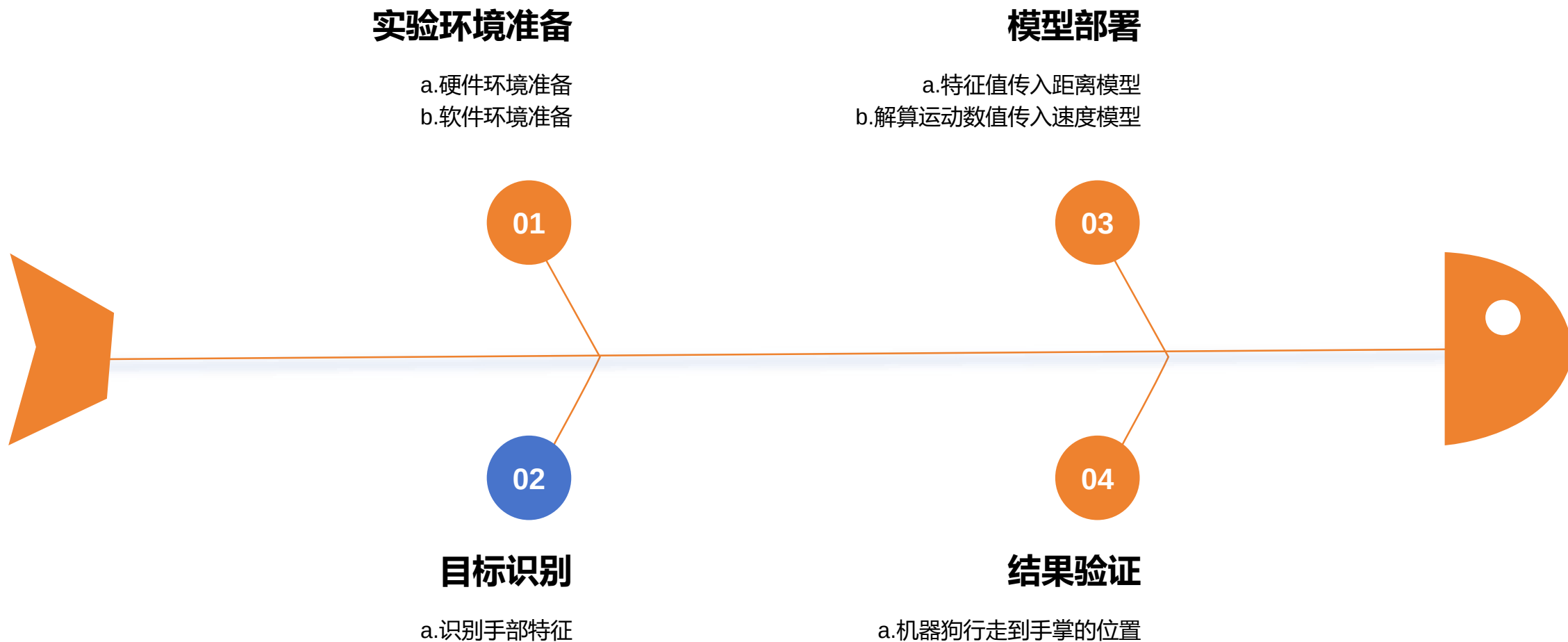


当机器狗通过摄像头识别到完整手掌后，随着人的手掌的移动，机器狗会朝手掌移动的方向行走，即手掌跟踪。

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (3学时)
- 六、模型应用及其部署 (4学时)
- 七、运行示例

2.实验过程描述



目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

3.1 硬件环境准备

3.2 机器狗远程连接

四、工程文件目录

五、数据工程 (3学时)

六、模型应用及其部署 (4学时)

七、运行示例

3.1 硬件环境准备

(1) SD卡中系统烧录

向SD卡中烧录镜像的步骤如下：

- ① 将SD卡插入读卡器，再将读卡器插到PC上。



3.1 硬件环境准备

(1) SD卡中系统烧录

②在浏览器中进入Ascend官网，选择“产品”->“开发者套件”



3.1 硬件环境准备

(1) SD卡中系统烧录

③点击“资源”，进入资源下载界面



3.1 硬件环境准备

(1) SD卡中系统烧录

④选择“制卡工具下载”，下载完成后进行安装。

寻你所需，玩转AI开发_

快速入门



准备

请提前做好下列物品，或查看[硬件准备清单](#)，快速上手不卡顿

- MicroSD卡（推荐64GB）及读卡器（可选）
- 获取制卡工具，一键烧录镜像完成开发环境准备

[制卡工具下载](#) [校验文件下载](#)



入门

提供入门 Atlas 200I DK A2 的全流程指导，帮助你30分钟内轻松完成从启动开发者套件到运行首样例的过程

[快速开始（课程）](#) [快速开始（文档）](#)



参考工具

提供Atlas 200I DK A2 进一步开发所需的参考工具及配套资源，可按需下载

模型适配工具

使能开发者0代码完成数据标注、模型训练、模型部署到昇腾开发者套件

[软件包下载](#) [校验文件下载](#)

配套Conda环境包

模型适配工具运行所必须的配套conda依赖环境，需要提前安装conda并导入conda环境包

[软件包下载](#) [校验文件下载](#)

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Page 23

 HUAWEI

3.1 硬件环境准备

(1) SD卡中系统烧录

⑤打开镜像烧录工具Ascend AI Devkit Imager, 选择Atlas 200I DK A2



3.1 硬件环境准备

(1) SD卡中系统烧录

⑥打开“配置网络信息”，选择“Type-C”选项卡。可以看到Type-C接口默认IP为192.168.0.2。选择默认IP。

镜像网络信息配置

×

以下为镜像默认IP信息，您可以根据PC/路由器IP信息重新填写网络信息

网口信息

ETH0

ETH1

Type-C

☐ 自动获取IP地址

☒ 使用下面的IP地址

IP地址

192 · 168 · 0 · 2

子网掩码

255 · 255 · 255 · 0

默认网关

· · ·

首选DNS服务器

8 · 8 · 8 · 8

备用DNS服务器

114 · 114 · 114 · 114

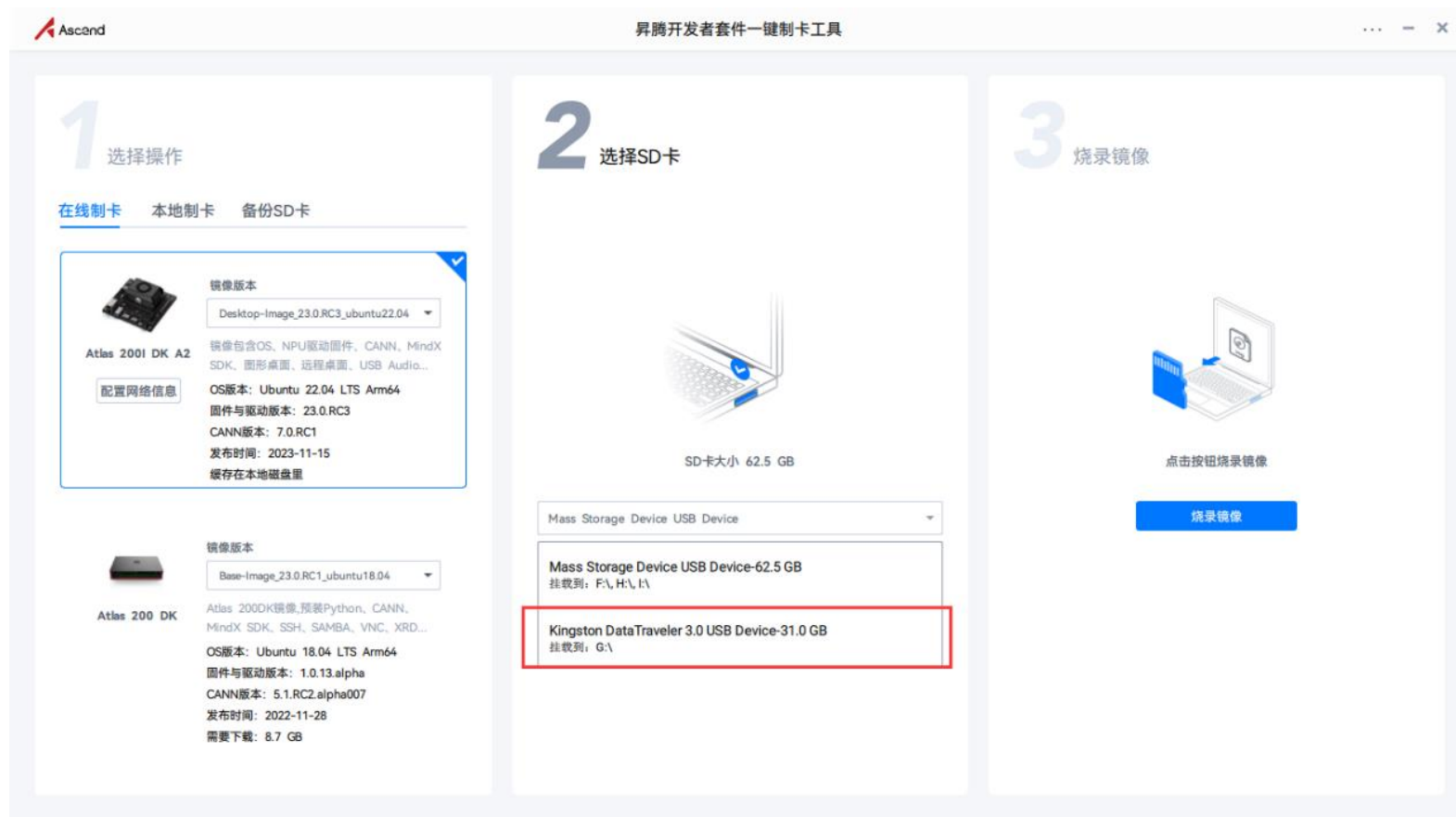
确定

取消

3.1 硬件环境准备

(1) SD卡中系统烧录

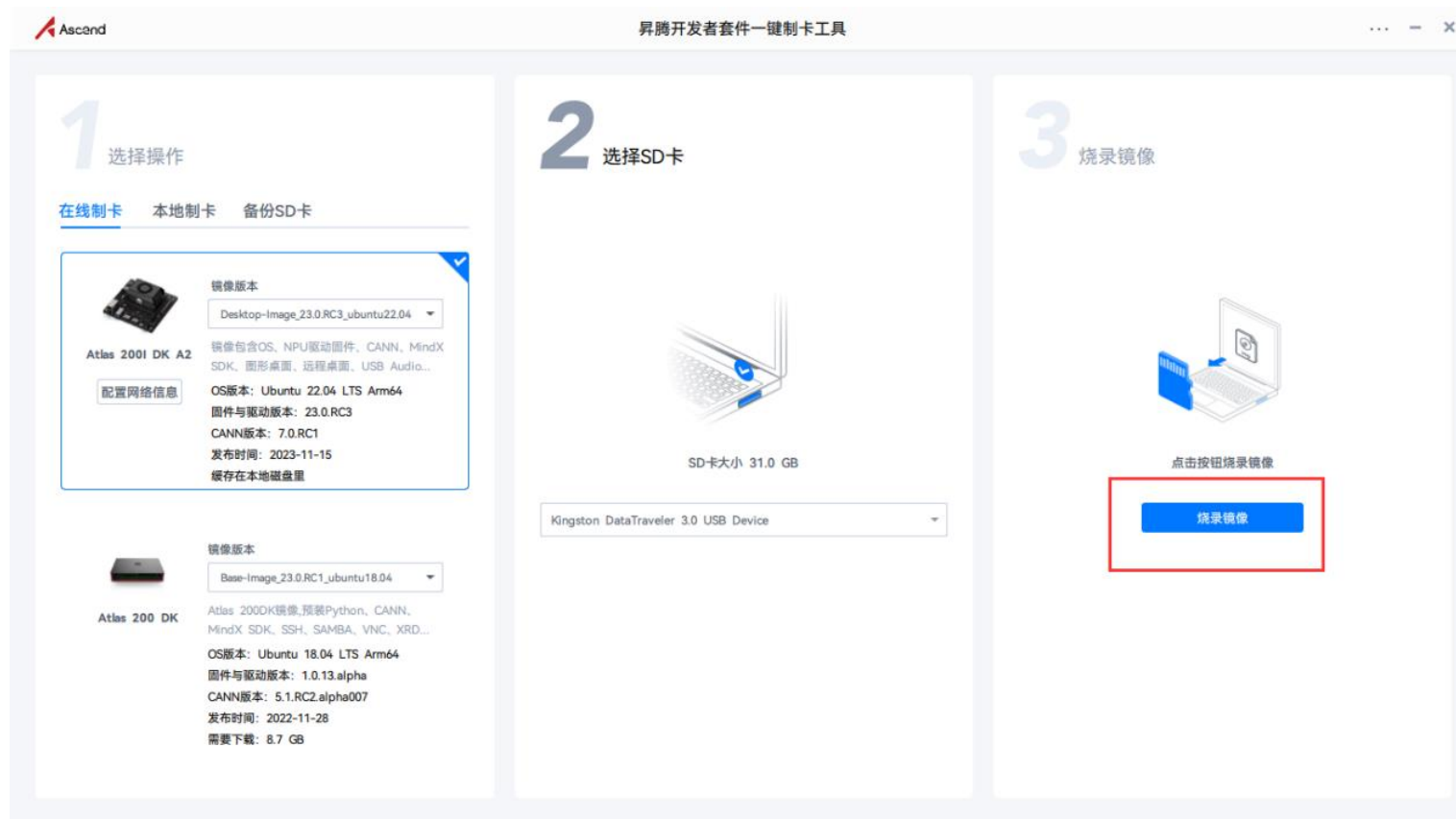
⑦选择要烧录的SD卡。



3.1 硬件环境准备

(1) SD卡中系统烧录

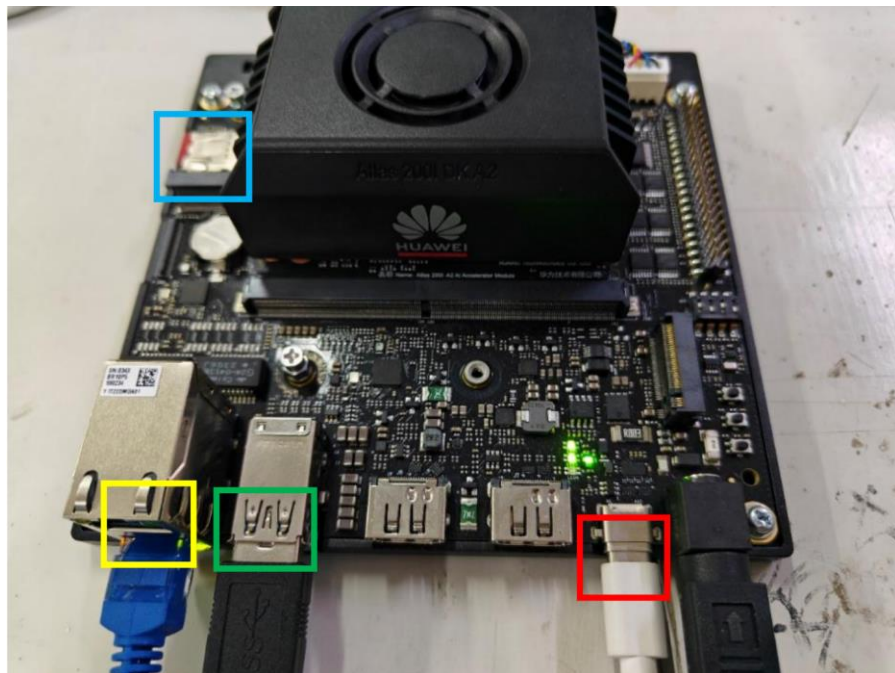
⑧点击“烧录镜像”，等待镜像烧录完成。



3.1 硬件环境准备

(2) 开发板连接

将烧录好系统的SD卡插入到开发板中（下方蓝色框中），昇腾开发板和PC机通过Type-C线连接，将Type-C线和电源线连接到开发板（下方红色框中），并将海康工业相机和机器狗的USB线连接到开发板（下方绿色框中）。同时可以连接网线（下方黄色框中），让开发版连接互联网，方便环境依赖包的下载。连接方式如下图所示。



3.1 硬件环境准备

(3) PC端远程连接开发板

当开发者套件通过Type-C接口和PC连接时，如何在PC将接口 IP地址修改为和开发者套件接口 IP地址同一个网段（如开发者套件Type-C 接口为192.168.0.2，PC对应USB或Type-C接口为192.168.0.101），并使用SSH工具远程登录开发者套件。

1) 安装Windows的USB网卡驱动（以Windows 10系统为例）

①在PC上右键单击“此电脑”，选择“更多”→“管理”，进入“计算机管理”界面。

②在“计算机管理”操作界面中选择“设备管理器→“其他设备”，如图所示，RNDIS为未识别状态。

注意：如果未出现RNDIS，请按以下方法排查：

确保开发者套件Type-C接口和PC已通过连接线正常连接。

更换具备数据传输能力的Type-C数据线（一般手机充电线拥有数据传输能力），继续查看是否出现RNDIS。

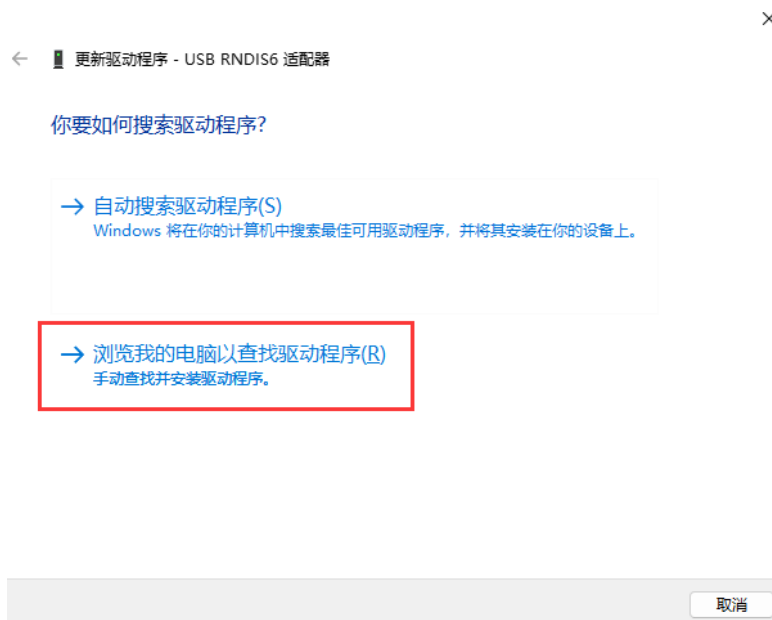
3.1 硬件环境准备

(3) PC端远程连接开发板

如果尝试现场所有Type-C数据线，仍无法出现RNDIS，则考虑使用 RJ45网线连接PC和开发者套件的以太网口的方式登录开发者套件。

③右键单击“RNDIS”，选择“更新驱动程序(P)”。

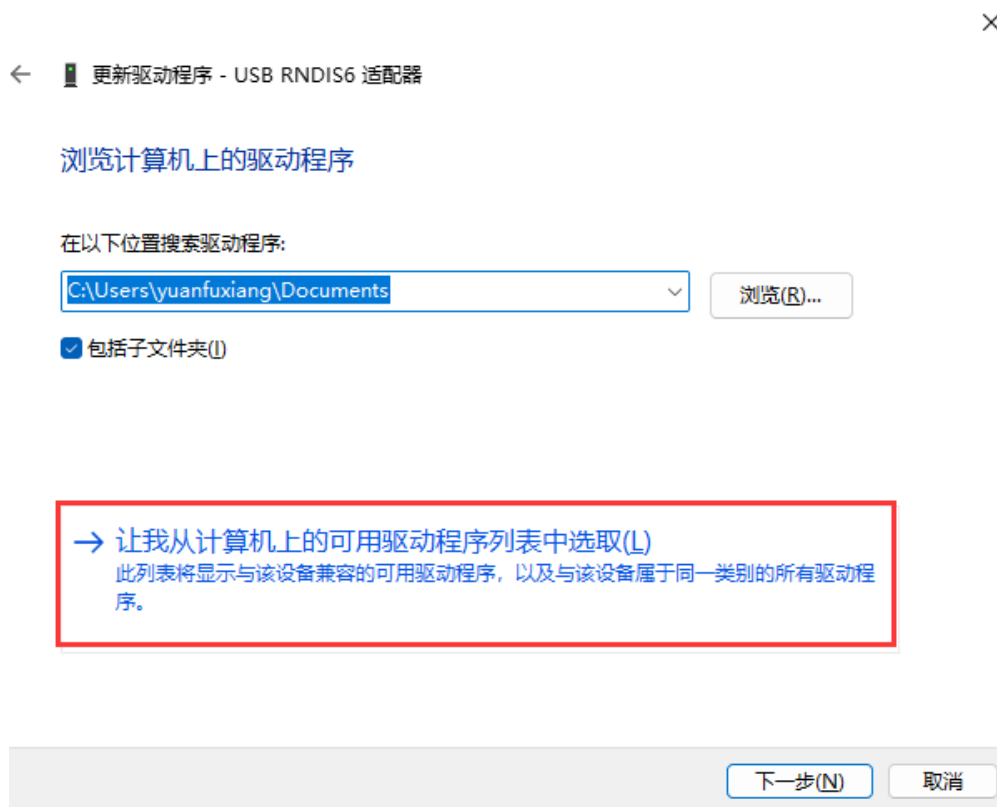
④在弹出的“更新驱动程序”→“RNDIS窗口”中选择“浏览我的计算机以查找驱动程序软件(R)”。



3.1 硬件环境准备

(3) PC端远程连接开发板

⑤然后选择“让我从计算机上的可用驱动程序列表中选择(L)”。



3.1 硬件环境准备

(3) PC端远程连接开发板

⑥在“常见硬件类型”列表中选择“网络适配器”，单击“下一页(N)”。



3.1 硬件环境准备

(3) PC端远程连接开发板

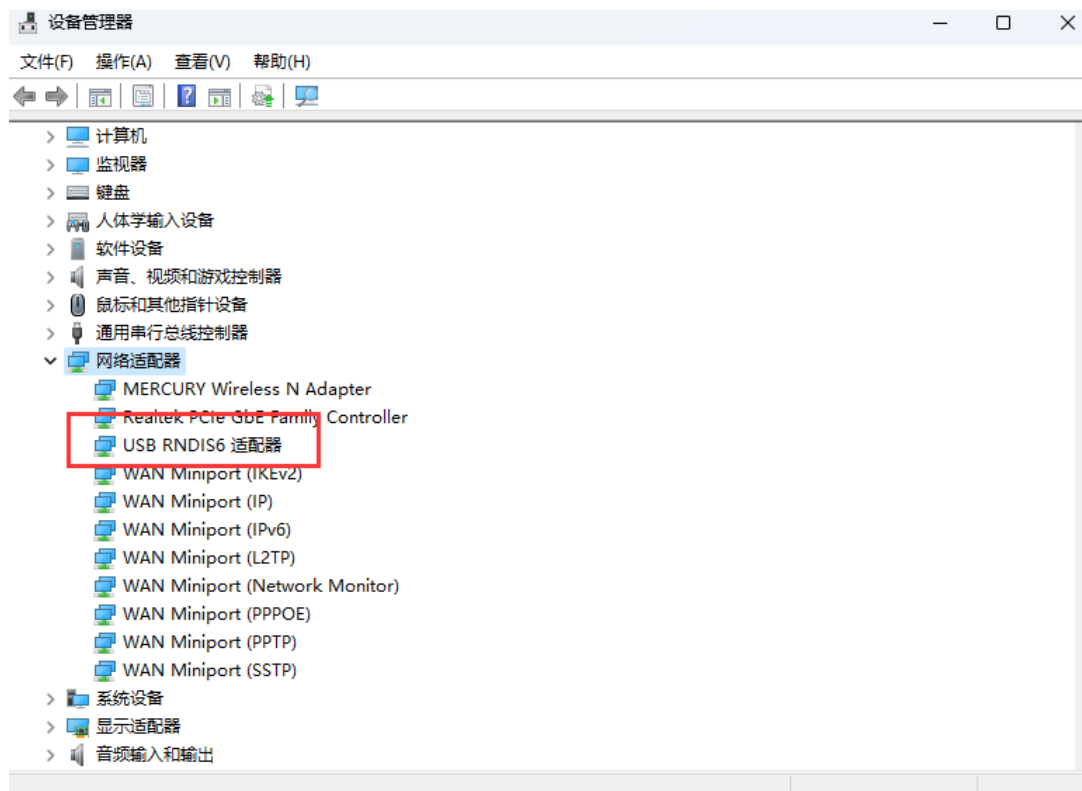
⑦在“选择要为此硬件安装的设备驱动程序”界面中选择“Microsoft”厂商的“USB RNDIS6适配器”。



3.1 硬件环境准备

(3) PC端远程连接开发板

- ⑧单击“下一页”，在弹出的“更新驱动程序警告”窗口选择“是”。
- ⑨返回“设备管理器”→“网络适配器”，可看到已经正常显示了USB RNDIS6适配器的驱动。



3.1 硬件环境准备

(4) 配置PC接口IP地址

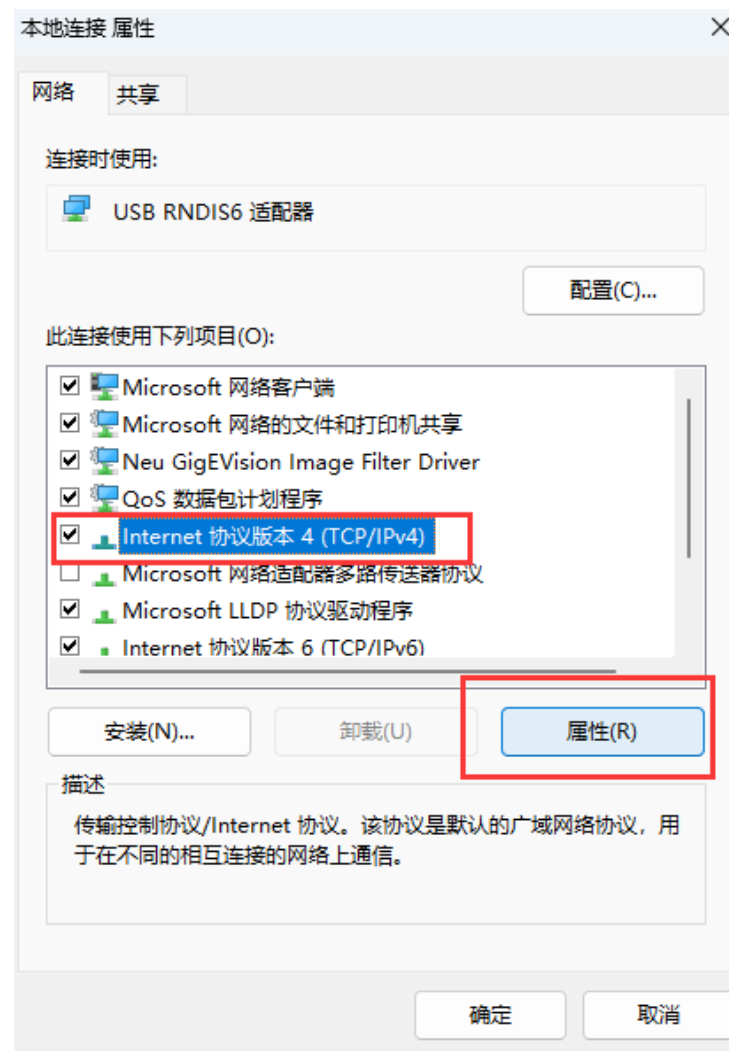
本步骤以 Windows 10 系统为例。

- ①在 PC 上打开 “开始” ， 单击 “设置” 按钮， 进入 “Windows 设置” 界面。
- ②选择 “网络和 Internet” ， 单击 “更改适配器选项” 。
- ③鼠标右键单击 “本地连接” 后鼠标左键单击 “属性” 进入 “本地连接 属性” 界面（使用 Type-C 接口连接时一般为 “本地连接 x” ， x 为数字， 以实际显示的数字为准。

3.1 硬件环境准备

(4) 配置PC接口IP地址

④选择 “Internet协议版本4 (TCP/IPv4) ” ， 单击 “属性” 。



3.1 硬件环境准备

(4) 配置PC接口IP地址

⑤勾选“使用下面的IP地址”选项，填写IP地址（图示以192.168.0.101为例）和子网掩码，默认网关与DNS服务器地址为空，单击“确定”保存。

Internet 协议版本 4 (TCP/IPv4) 属性

常规

如果网络支持此功能，则可以获取自动指派的 IP 设置。否则，你需要从网络系统管理员处获得适当的 IP 设置。

☐ 自动获得 IP 地址(O)

☒ 使用下面的 IP 地址(S):

IP 地址(I): 192 . 168 . 0 . 101

子网掩码(U): 255 . 255 . 255 . 0

默认网关(D): . . .

☐ 自动获得 DNS 服务器地址(B)

☒ 使用下面的 DNS 服务器地址(E):

首选 DNS 服务器(P): . . .

备用 DNS 服务器(A): . . .

☐ 退出时验证设置(L)

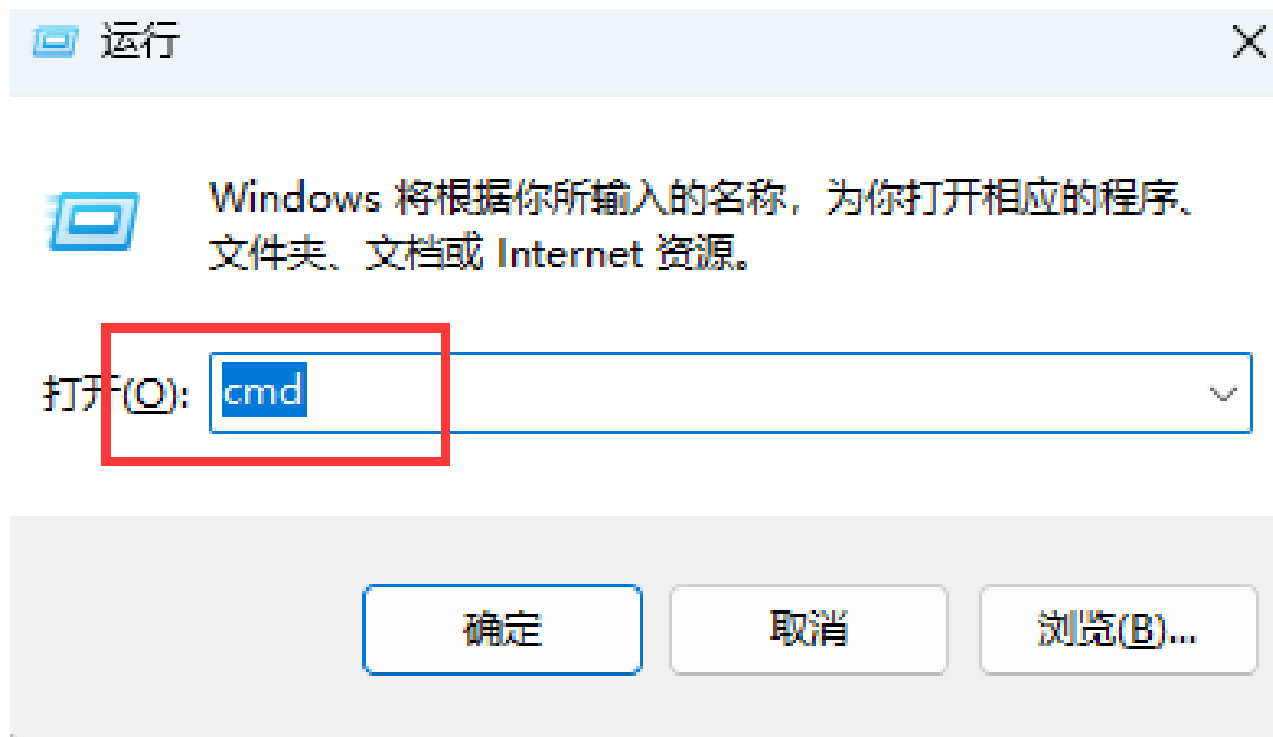
高级(V)...

确定 取消

3.1 硬件环境准备

(4) 配置PC接口IP地址

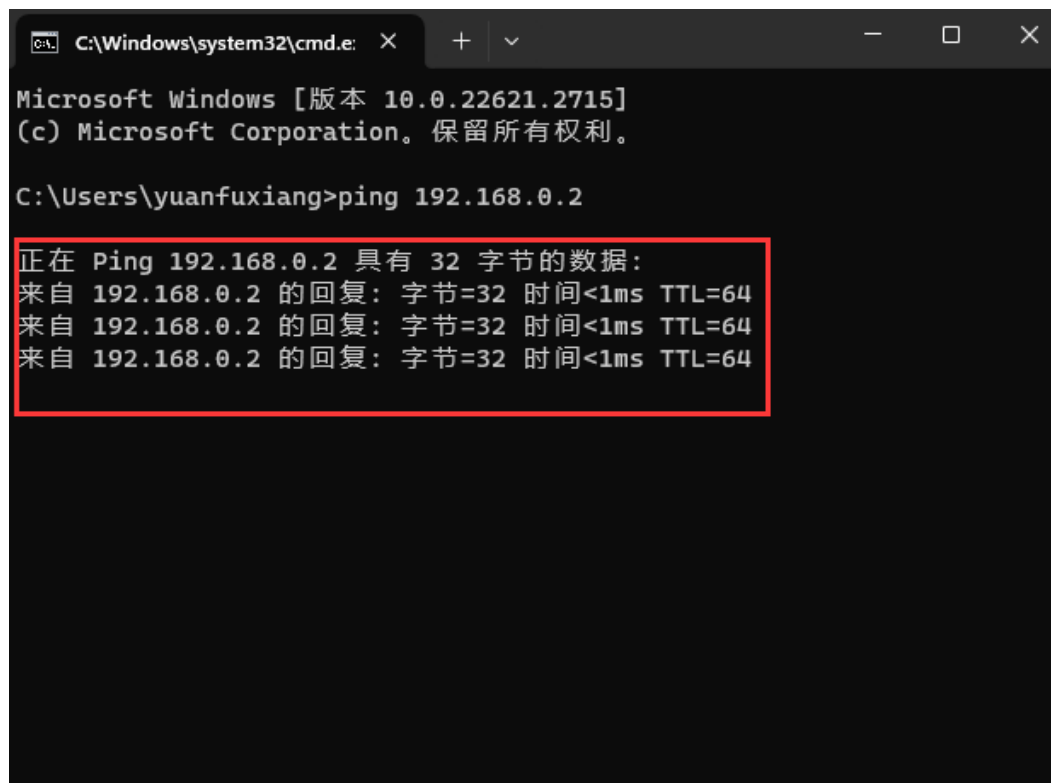
⑥使用快捷键 “Win+R” ， 在运行窗口输入cmd进入命令行窗口。



3.1 硬件环境准备

(4) 配置PC接口IP地址

⑦输入ping 192.168.0.2命令，查看是否能够访问开发板IP。



```
C:\Windows\system32\cmd.exe X + - □ X

Microsoft Windows [版本 10.0.22621.2715]
(c) Microsoft Corporation。保留所有权利。

C:\Users\yuanfuxiang>ping 192.168.0.2

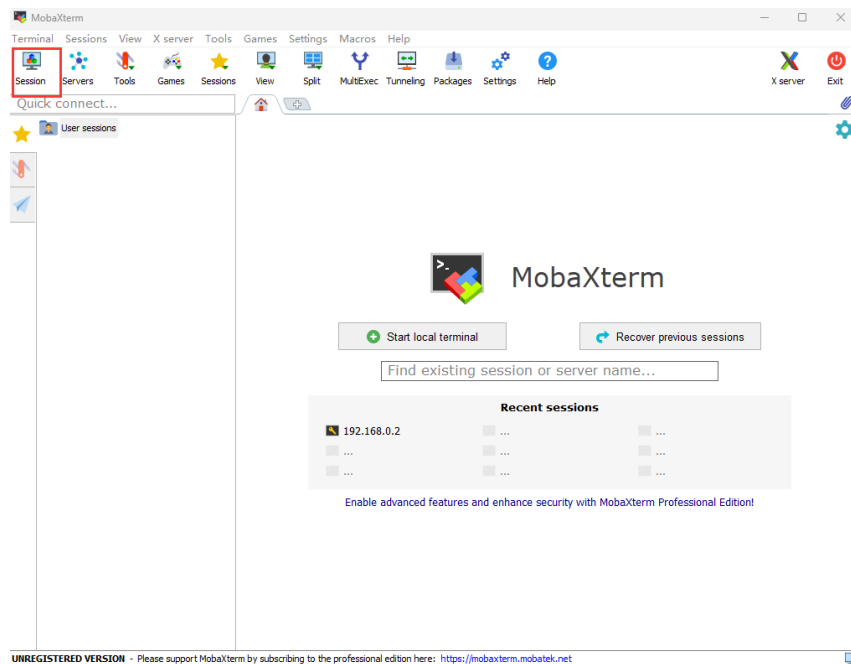
正在 Ping 192.168.0.2 具有 32 字节的数据:
来自 192.168.0.2 的回复: 字节=32 时间<1ms TTL=64
来自 192.168.0.2 的回复: 字节=32 时间<1ms TTL=64
来自 192.168.0.2 的回复: 字节=32 时间<1ms TTL=64
```

3.1 硬件环境准备

(5) 使用ssh远程登录开发板

本文使用MobaXterm工具在PC上远程登录开发板，单击下载链接获取MobaXterm软件压缩包，解压获得“MobaXterm_Personal_22.2.exe”。

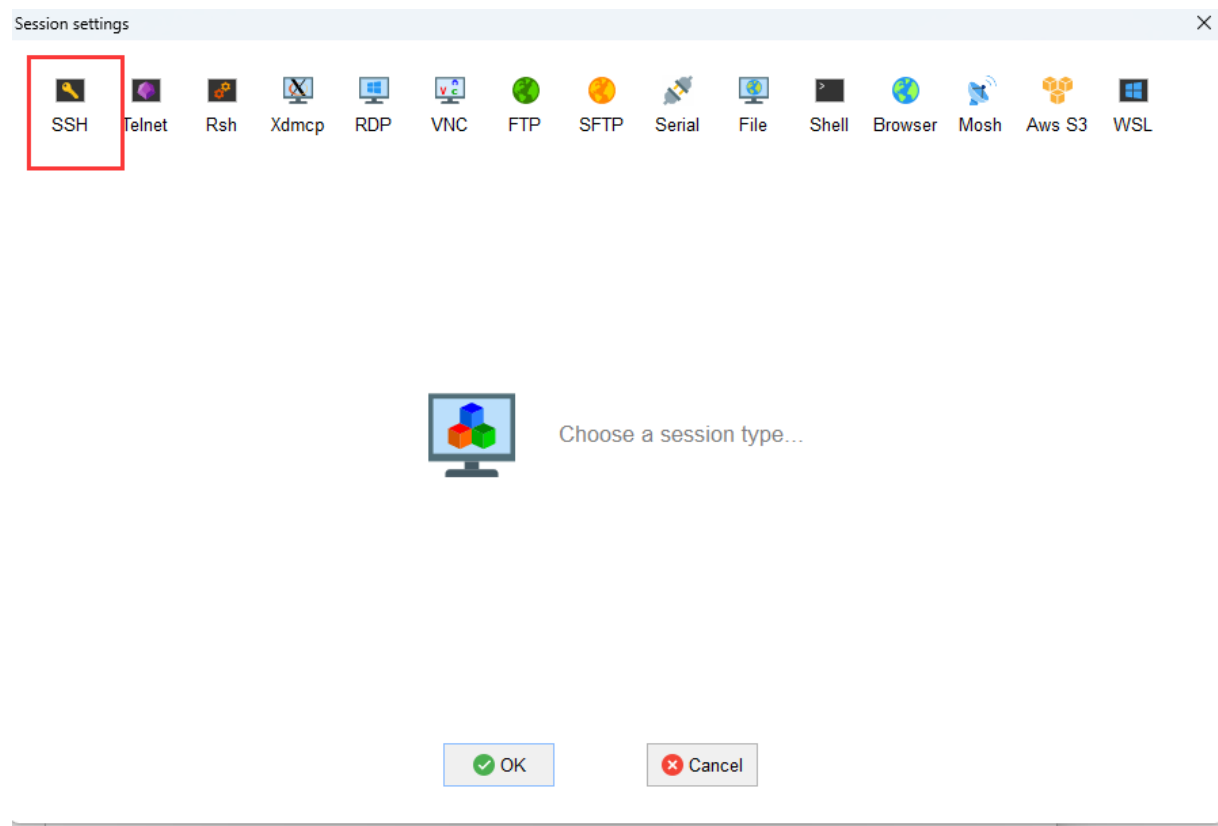
- ①双击“MobaXterm_Personal_22.2.exe”启动SSH登录工具。
- ②单击左上方的“Session”进入界面。



3.1 硬件环境准备

(5) 使用ssh远程登录开发板

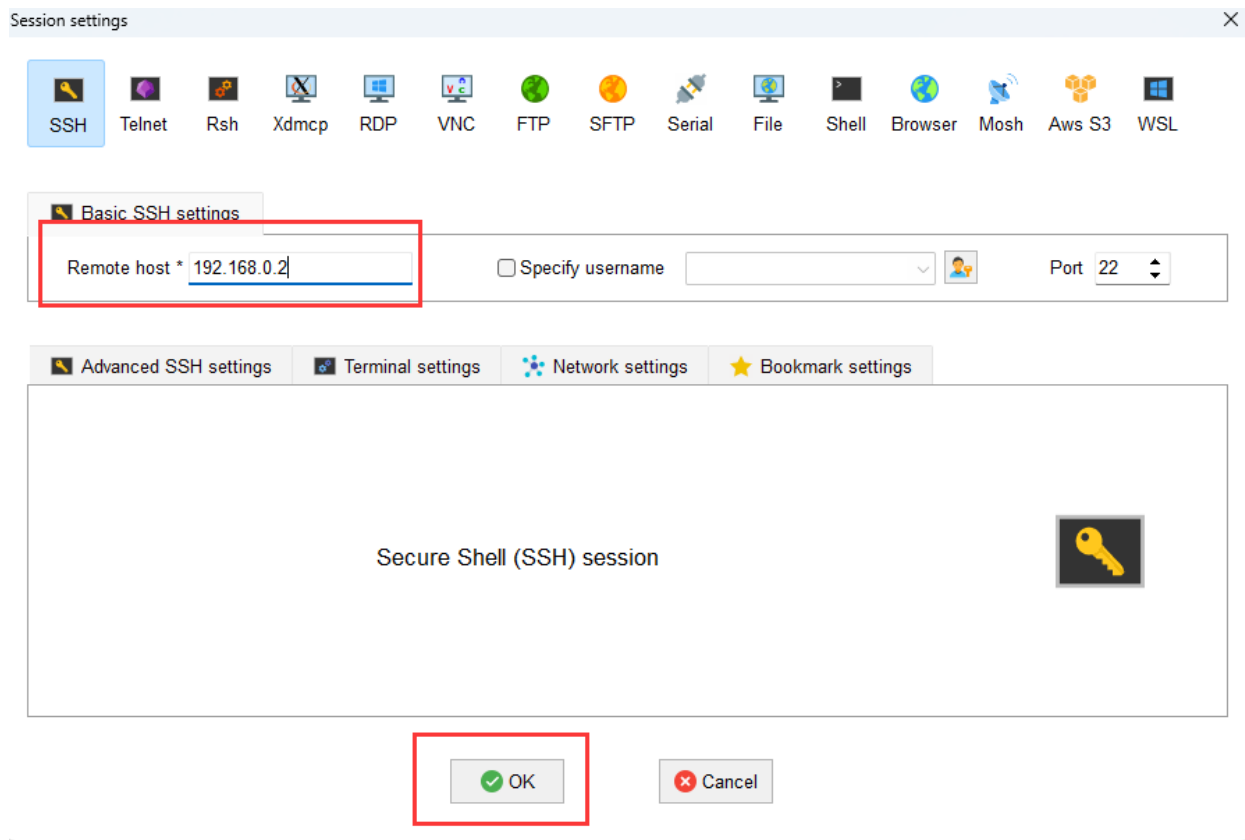
③单击左上方的“SSH”进入 SSH 连接配置界面



3.1 硬件环境准备

(5) 使用ssh远程登录开发板

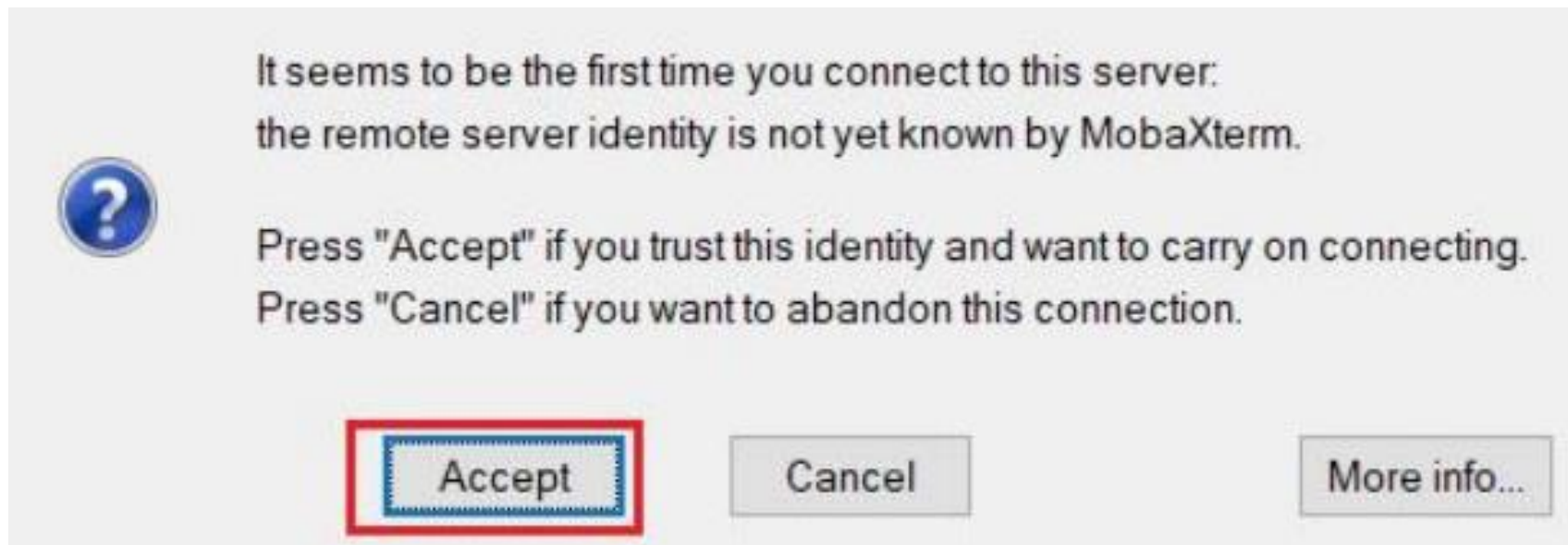
④根据硬件连接方式填写开发者套件连接PC的接口实际IP地址（一键制卡中配置的IP地址）。图示以192.168.0.2为例，请根据实际修改。



3.1 硬件环境准备

(5) 使用ssh远程登录开发板

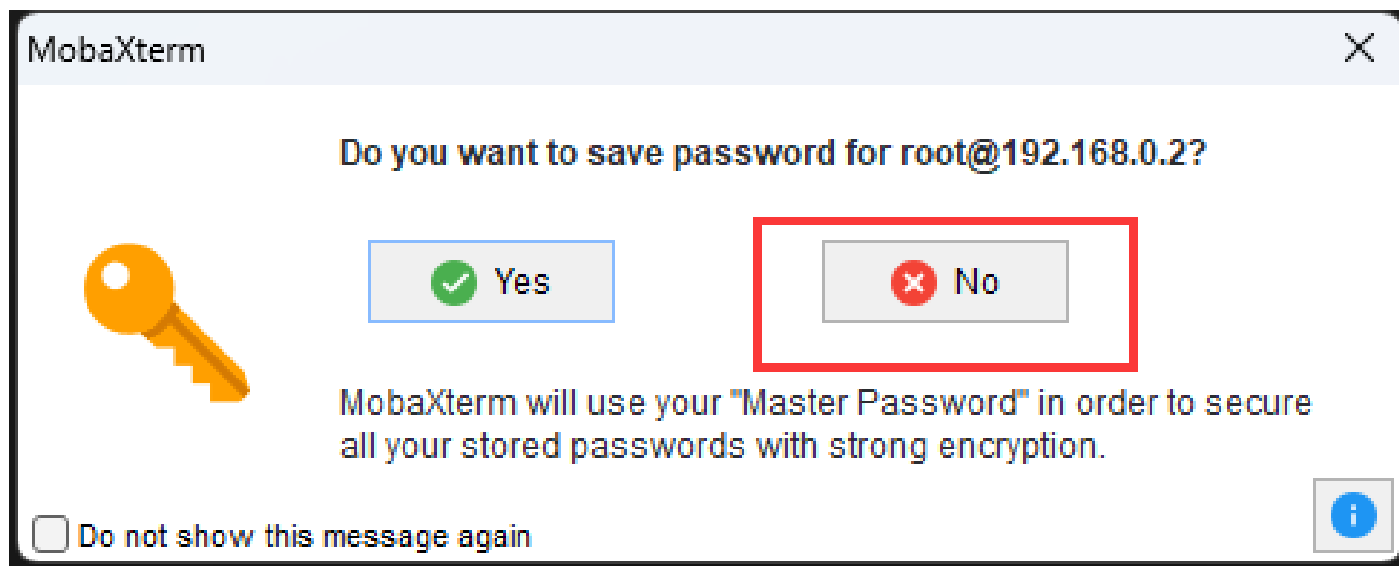
⑤单击“OK”按钮，首次连接开发者套件时，SSH工具提示是否信任连接的设备，单击“Accept”。



3.1 硬件环境准备

(5) 使用ssh远程登录开发板

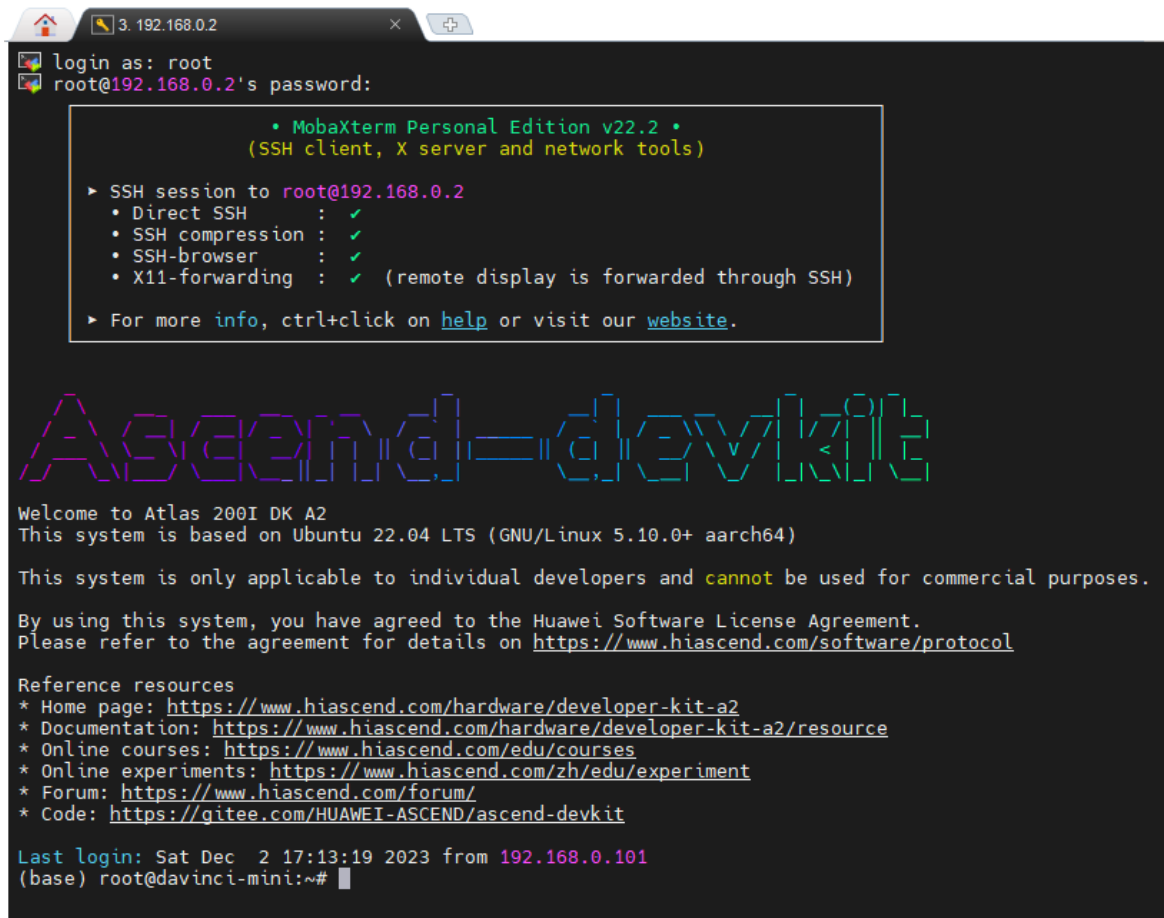
⑥进入远程登录界面后，输入 root 用户名登录密码（默认为 Mind@123）登录开发者套件，请修改默认密码，并妥善保管新密码。SSH 工具界面会出现保存密码提示，可以单击 “No”，不保存密码直接登录开发者套件。



3.1 硬件环境准备

(5) 使用ssh远程登录开发板

⑦远程登录开发者套件成功界面如图所示。



```
login as: root
root@192.168.0.2's password:

• MobaXterm Personal Edition v22.2 •
  (SSH client, X server and network tools)

▶ SSH session to root@192.168.0.2
  • Direct SSH      : ✓
  • SSH compression : ✓
  • SSH-browser     : ✓
  • X11-forwarding  : ✓ (remote display is forwarded through SSH)

▶ For more info, ctrl+click on help or visit our website.

Ascend=devkit

Welcome to Atlas 200I DK A2
This system is based on Ubuntu 22.04 LTS (GNU/Linux 5.10.0+ aarch64)

This system is only applicable to individual developers and cannot be used for commercial purposes.

By using this system, you have agreed to the Huawei Software License Agreement.
Please refer to the agreement for details on https://www.hiascend.com/software/protocol

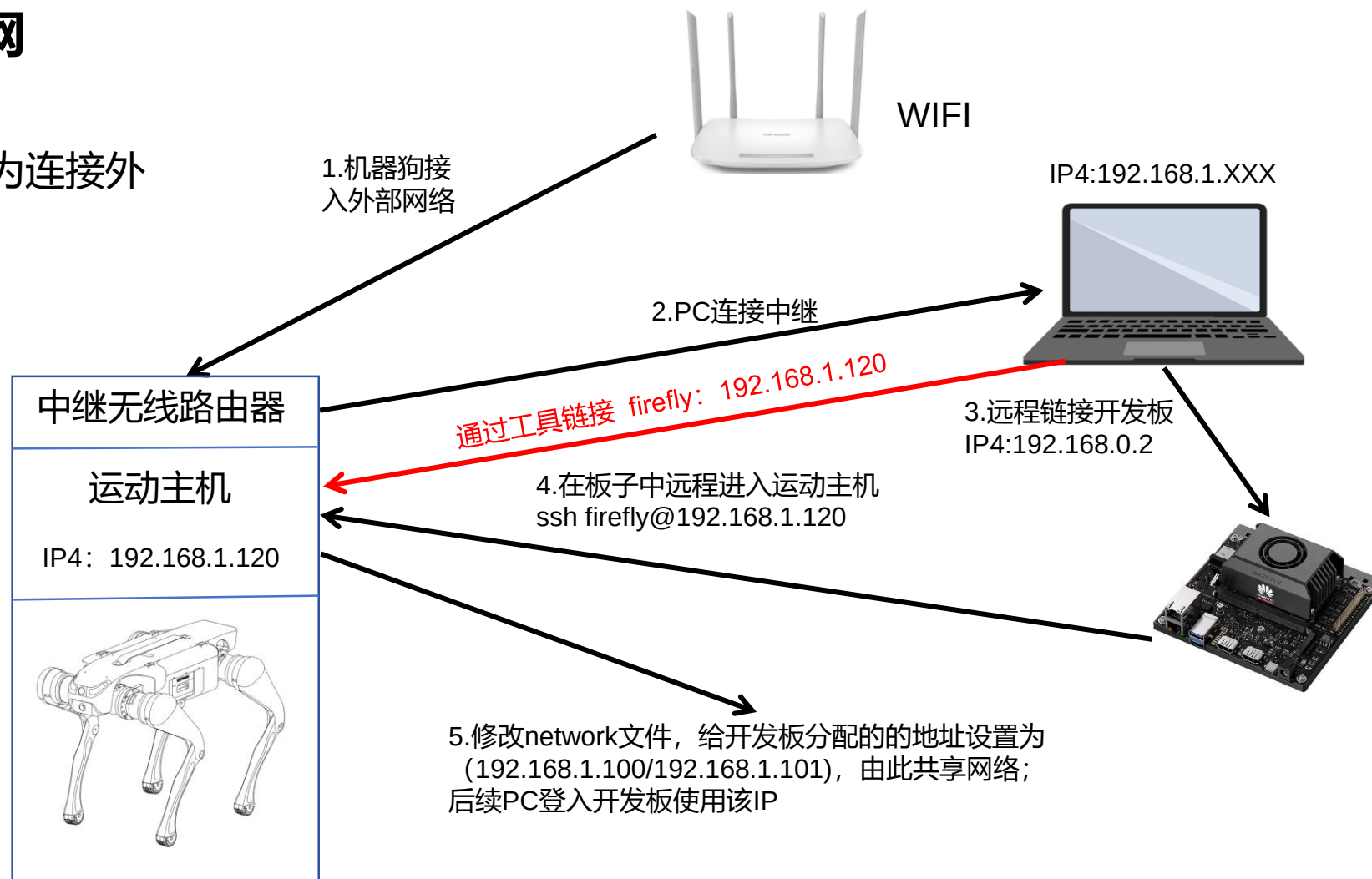
Reference resources
* Home page: https://www.hiascend.com/hardware/developer-kit-a2
* Documentation: https://www.hiascend.com/hardware/developer-kit-a2/resource
* Online courses: https://www.hiascend.com/edu/courses
* Online experiments: https://www.hiascend.com/zh/edu/experiment
* Forum: https://www.hiascend.com/forum/
* Code: https://gitee.com/HUAWEI-ASCEND/ascend-devkit

Last login: Sat Dec  2 17:13:19 2023 from 192.168.0.101
(base) root@davinci-mini:~#
```

3.1 硬件环境准备

(6) 机器狗中继连接外网

此步骤是为了让单机环境变为连接外部网络的环境



3.1 硬件环境准备

(6) 机器狗中继连接外网

注：本教程主要为暂时没有外置无线网卡的用户提供连接外网方案，教程以中继手机热点为例进行说明，中继无线路由器同理。

1、电脑连接 机器狗的wifi，点击电脑右下角的wifi图标，点击属性查看IPv4 DNS服务器的ip地址。如下图所示。

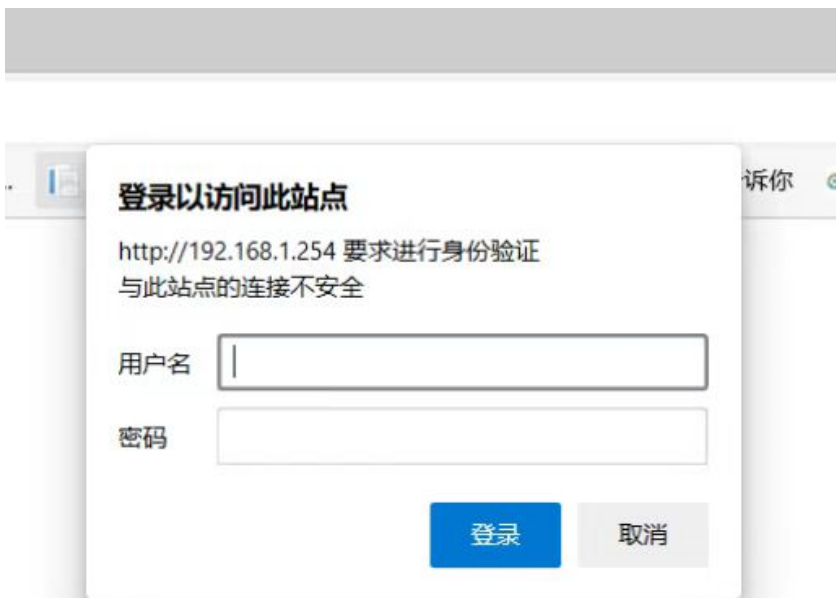


属性	
SSID:	YSC-JYML-emzqzj
协议:	Wi-Fi 4 (802.11n)
安全类型:	WPA2-个人
网络频带:	2.4 GHz
网络通道:	11
链接速度(接收/传输):	144/144 (Mbps)
IPv4 地址:	192.168.1.101
IPv4 DNS 服务器:	192.168.1.254 8.8.8.8
制造商:	Intel Corporation
描述:	Intel(R) Wireless-AC 9560 160MHz
驱动程序版本:	22.200.0.6
物理地址(MAC):	
复制	

3.1 硬件环境准备

(6) 机器狗中继连接外网

2、打开电脑浏览器，输入IPv4 DNS服务器的ip，默认登录用户和密码均为admin。
如下图所示。

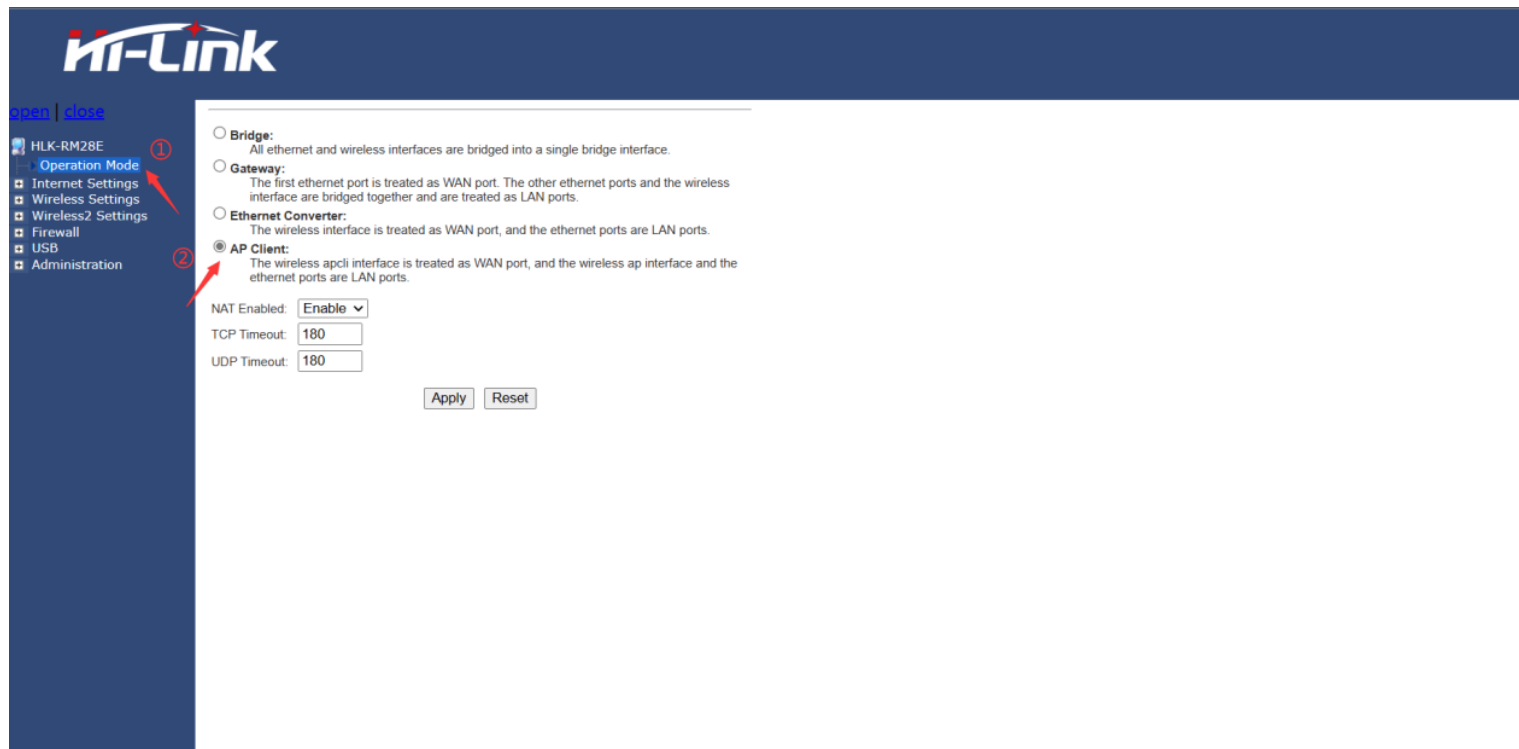


3.1 硬件环境准备

(6) 机器狗中继连接外网

3、打开手机热点。

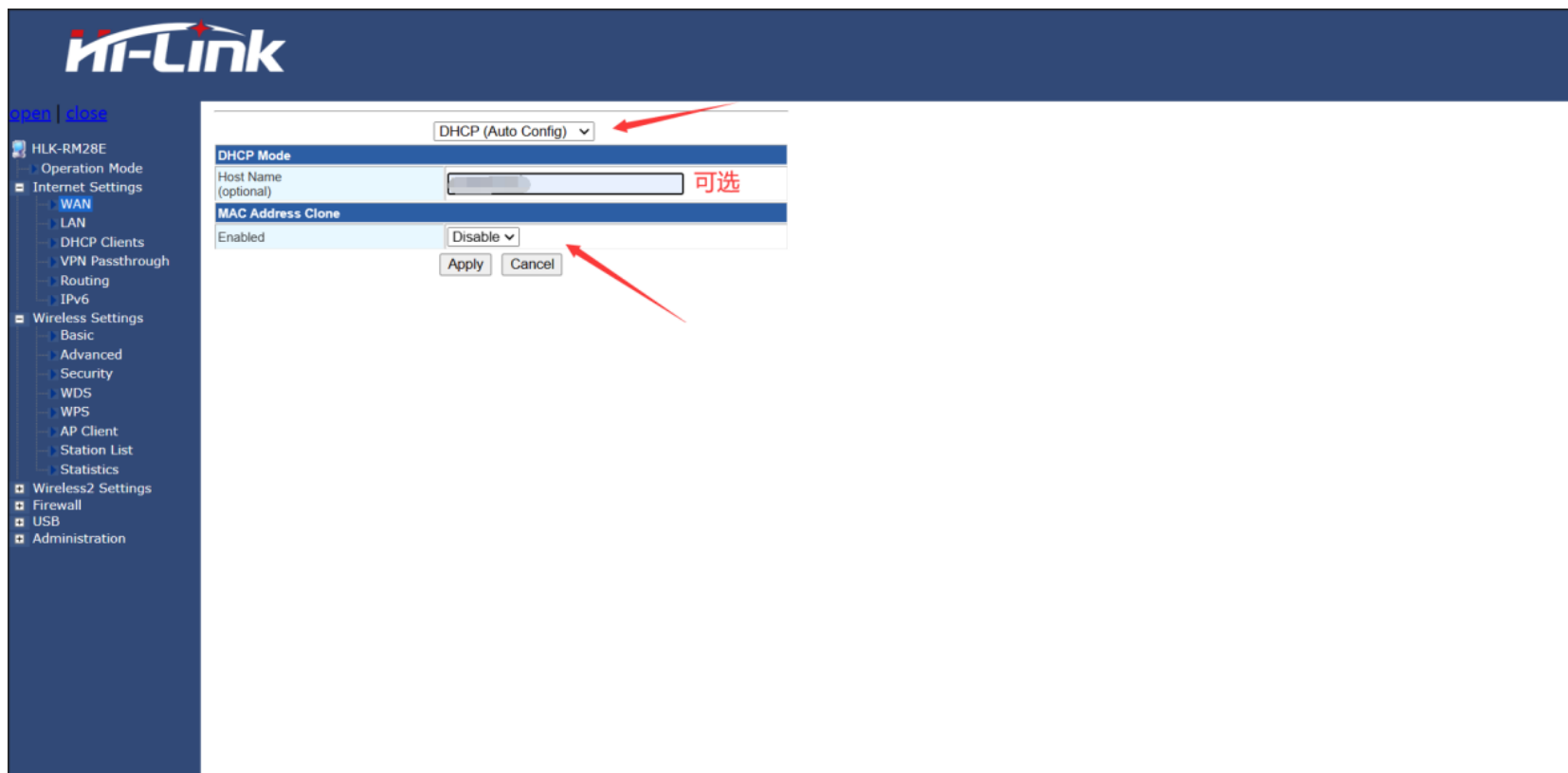
4、进入Hi-link设置页面，点击Operation Mode，选择AP Client，最后点击Apply。如下图所示。



3.1 硬件环境准备

(6) 机器狗中继连接外网

5、展开Internet Settings，点击WAN，参照下图进行设置，最后点击Apply。



3.1 硬件环境准备

(6) 机器狗中继连接外网

6、展开Wireless Settings，点击AP Client，查看site Survey表中是否有自己的手机热点名称，如若没有点击SCAN进行扫描，在AP client Parameters表中参照下图进行设置，最后点击Apply。

HLK-RM28E

Operation Mode

Internet Settings

Wireless Settings

- Basic
- Advanced
- Security
- WDS
- WPS
- AP Client
- Station List
- Statistics

Wireless2 Settings

Firewall

USB

Administration

AP Client Feature

You could configure AP Client parameters here.

AP Client Parameters

SSID

HelloWorld

MAC Address (Optional)

Security Mode

WPA2PSK

Encryption Type

AES

Pass Phrase

Apply

Cancel

SCAN

Site Survey

Ch	SSID	BSSID	Security	Signal(%)	W-Moe	ExtCh	NT
1	weichao	f4 84 8d 97 88 fc	WPA1PSKWPA2PSK/AES	68	11b/g/n	ABOVE	In
1	weichao	f4 84 8d 97 8f c2	WPA1PSKWPA2PSK/AES	70	11b/g/n	ABOVE	In
1	KRN1303	10 5d dc 2f a4 20	WPA1PSKWPA2PSK/AES	50	11b/g/n	NONE	In
1	iTV-X1oT	aa 00 74 e6 e3 46	WPAPSK/TKIPAES	52	11b/g/n	NONE	In
1	jsscsc	ec 60 73 e4 fc 3e	WPA1PSKWPA2PSK/AES	39	11b/g/n	ABOVE	In
1		ee 60 73 74 fc 3e	WPA1PSKWPA2PSK/AES	39	11b/g/n	ABOVE	In
1	0xE997B9E5A587E69CBAE599A8E4BABA	a2 2e 55 9a b0 38	WPA1PSKWPA2PSK/AES	100	11b/g/n	NONE	In
1	ChinaNet-X1oT	9a 00 74 e6 e3 46	WPAPSK/TKIPAES	52	11b/g/n	NONE	In
1	weichao	f4 84 8d 97 8b ab	WPA1PSKWPA2PSK/AES	100	11b/g/n	ABOVE	In
3	Baiyuan	b8 f0 b9 53 b0 f8	WPA1PSKWPA2PSK/AES	100	11b/g/n	ABOVE	In
4	409-DT-2G	80 3f 5d a9 ac 7e	WPA2PSK/AES	65	11b/g/n	ABOVE	In
4	ChinaNet-wmwX	b0 b1 94 ef d2 c3	WPAPSK/TKIPAES	68	11b/g/n	NONE	In
5	ChinaNet-zu2d	14 30 04 00 5d 84	WPA1PSKWPA2PSK/TKIPAES	63	11b/g/n	NONE	In
6	ChinaNet-WrQ4	2c 1a 01 d1 ea 88	WPA1PSKWPA2PSK/TKIPAES	78	11b/g/n	NONE	In
6	weichao	f4 84 8d 97 8e cc	WPA1PSKWPA2PSK/AES	86	11b/g/n	BELOW	In
6		18 3c b7 37 74 99	WPA2PSK/AES	94	11b/g/n	NONE	In

① 输入手机的热点名称

② 可选

③ 输入热点加密类型

④ 输入手机的热点密码

3.1 硬件环境准备

(6) 机器狗中继连接外网

7、重启机器狗，注意中继时热点需一直处于开启状态，电脑连接机器狗的wifi，使用远程软件登录，此时机器狗处于连接外网状态。测试是否有网络，点击浏览器，搜索任意内容，如“百度学术”，能正常显示下图说明正常。



目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

3.1 硬件环境准备

3.2 机器狗远程连接

四、工程文件目录

五、数据工程 (3学时)

六、模型应用及其部署 (4学时)

七、运行示例

3.2 机器狗远程连接

用户可通过SSH 远程连接到运动主机。

将开发主机连接到
机器人WiFi。

远程连接运动主机

在开发主机上打开SSH 连
接软件

输入ssh
firefly@192.168.1.120
密码为 firefly,
或者直接通过SSH软件进入主
机

进入jy_exe, 并打
开配置文件

```
cd  
/home/firefly/jy_ex  
e/conf/
```

```
vim network.toml
```

配置文件
network.toml 内容

```
ip =  
'192.168.1.102'  
target_port = 43897  
local_port = 43893
```

修改配置文件

如果代码在机器人运动主机
内运行, IP 设置为运动主
机IP: 192.168.1.120;

如果代码在用户自己的开发主
机中运行, 设置为开发主机的
静态IP: 192.168.1.xxx;

重启运动程序

```
cd  
/home/firefly/jy_e  
xe  
sudo ./stop.sh  
sudo ./restart.sh
```

目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

4.1 PC

4.2 Ascend

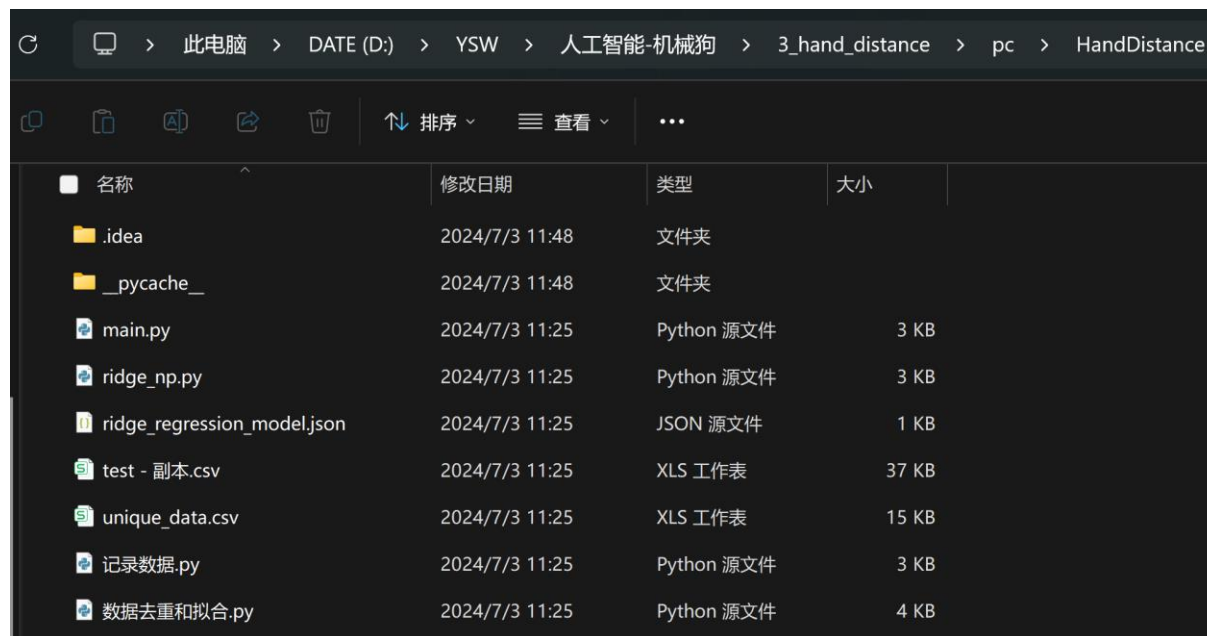
五、数据工程 (3学时)

六、模型应用及其部署 (4学时)

七、运行示例

4.1 PC

PC端yolov5的文件夹，
主要内容为dataset中存放的数据文件，
其他的地方不需要修改。
train.py是主要的训练文件



目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

4.1 PC

4.2 Ascend

五、数据工程 (3学时)

六、模型应用及其部署 (4学时)

七、运行示例

4.2 Ascend

开发板的文件夹总名叫做‘HandTracking
Command文件夹中的udp_command.py文件的
用来存放指令码

sendcommand文件夹中的heartbeat.py是发送心跳包，SendToCommand.py是发送指令

socketnetwork文件夹中的network_utils.py是与
机器狗建立通信的脚本

speeds文件夹中的sportspeed.py文件是机器狗
运动的速度方程

threading_utils文件夹中的ThreadTemplates.py
文件是线程

camera文件夹中放的是摄像头的使用文件
robotstatuswatcher文件夹中使用来监听机器狗
状态的

HanDistance文件夹中存放的是方程模型与采集
数据和分析数据的脚本。

```
(base) root@davinci-mini:/opt/lab/HandTracking# tree -I "__pycache__|*.pyc" -L 10
.
├── HandDistance
│   ├── main.py
│   ├── ridge_np.py
│   ├── ridge_regression_model.json
│   ├── test - 副本.csv
│   ├── unique_data.csv
│   ├── 数据去重和拟合.py
│   └── 记录数据.py
├── camera
│   ├── HKcamera.py
│   └── det_utils.py
├── command
│   ├── __init__.py
│   └── udp_command.py
├── hand_distance_main.py
├── robotstatuswatcher
│   ├── __init__.py
│   └── listener.py
├── sendcommand
│   ├── SendToCommand.py
│   ├── __init__.py
│   └── heartbeat.py
├── socketnetwork
│   ├── __init__.py
│   └── network_utils.py
├── speeds
│   ├── __init__.py
│   └── sportspeed.py
└── threading_utils
    └── ThreadTemplates.py
```

目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (3学时)

5.1配置开发板的HK驱动

5.2配置本地CVZone环境

5.3测试识别效果

5.4编写数据采集脚本

5.5采集手部特征值与数据格式化

5.6建立测距方程式模型

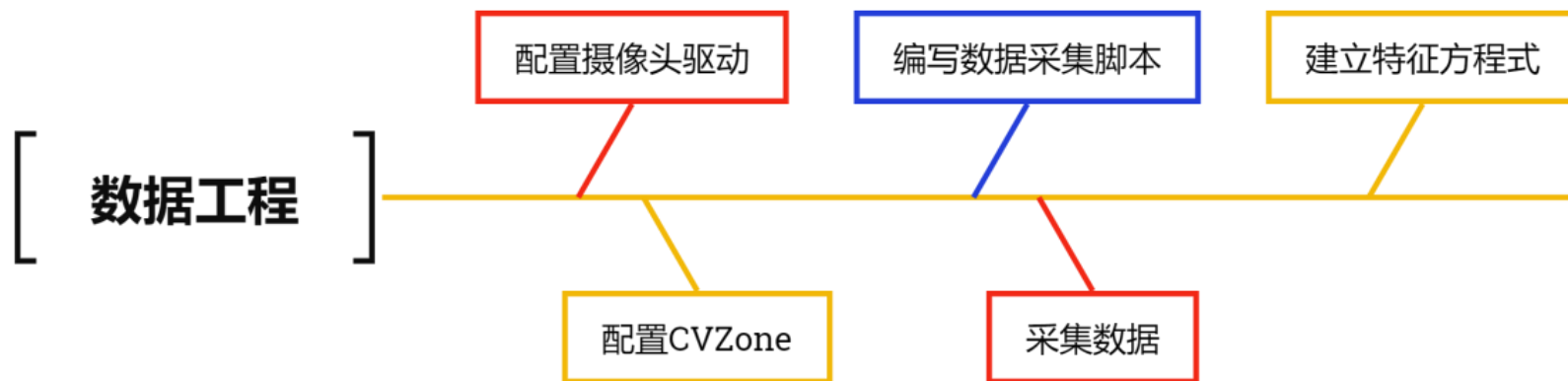
5.7建立角度方程式模型

5.8模型本地测试

六、模型应用及其部署 (4学时)

七、运行示例

五、数据工程



目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (3学时)

5.1配置开发板的HK驱动

5.2配置本地CVZone环境

5.3测试识别效果

5.4编写数据采集脚本

5.5采集手部特征值与数据格式化

5.6建立测距方程式模型

5.7建立角度方程式模型

5.8模型本地测试

六、模型应用及其部署 (4学时)

七、运行示例

5.1 配置开发板的HK驱动

Step 1

下载并安装MVS驱动文件 (Linux)
海康威视官网下载链接:
<https://www.hikrobotics.com/cn/machinevision/service/download?module=0>

Step 2

打开MobaXterm
远程控制，进入开发板安装驱动指令:

```
dpkg -i MVS-3.0.1_aarch64_20240422.deb
```

Step 3

在代码中使用需要加入包的路径:

```
sys.path.append("/opt/MVS/Samples/aarch64/Python/MvImport")from MvCameraControl_class import*
```

Step 4

使用摄像头的代码

```
python HKcamera.py
```

目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (3学时)

5.1配置开发板的HK驱动

5.2配置本地CVZone环境

5.3测试识别效果

5.4编写数据采集脚本

5.5采集手部特征值与数据格式化

5.6建立测距方程式模型

5.7建立角度方程式模型

5.8模型本地测试

六、模型应用及其部署 (4学时)

七、运行示例

5.2配置本地CVZone环境

安装cvzone相关的库：

```
pip install cvzone
```

如果出现其他的包缺失，按照报错提示安装即可

目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (3学时)

5.1配置开发板的HK驱动

5.2配置本地CVZone环境

5.3测试识别效果

5.4编写数据采集脚本

5.5采集手部特征值与数据格式化

5.6建立测距方程式模型

5.7建立角度方程式模型

5.8模型本地测试

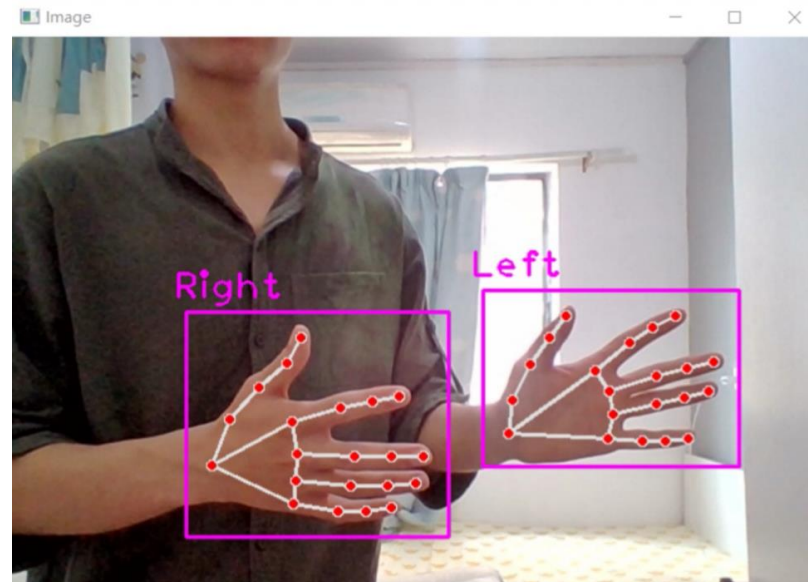
六、模型应用及其部署 (4学时)

七、运行示例

5.3测试识别效果

调用的测试代码如下：

```
1. import math
2. import cv2
3. from cvzone.HandTrackingModule import HandDetector
4. import numpy as np
5. import cvzone
6. import pandas as pd
7. import os
8. cap = cv2.VideoCapture(0)
9. cap.set(3, 640) # 设置帧的宽度
10. cap.set(4, 480) # 设置帧的高度
11. detector = HandDetector(detectionCon=0.8, maxHands=2)
12. while True:
13.     success, img = cap.read()
14.     if success: # 检查帧是否成功读取
15.         # 这个的 draw = False 是指只显示实际距离的检测框，手部的检测框不显示
16.         hands, img = detector.findHands(img)
17.         cv2.imshow("Image", img)
18.         # 按 'q' 键退出循环
19.         if cv2.waitKey(1) & 0xFF == ord('q'):
20.             break
21. cap.release()
22. cv2.destroyAllWindows()
```



目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (3学时)

5.1配置开发板的HK驱动

5.2配置本地CVZone环境

5.3测试识别效果

5.4编写数据采集脚本

5.5采集手部特征值与数据格式化

5.6建立测距方程式模型

5.7建立角度方程式模型

5.8模型本地测试

六、模型应用及其部署 (4学时)

七、运行示例

5.4编写数据采集脚本

我们可以编写一个简单的脚本输出数据看看长什么样子。

```
[398, 332, 0], [402, 312, -21], [416, 294, -34], [434, 295, -46], [
[398, 345, 0], [401, 332, -22], [412, 319, -36], [428, 320, -48], [
[380, 342, 0], [403, 360, -4], [433, 367, -9], [454, 373, -15], [46
[383, 342, 0], [404, 355, -3], [434, 362, -9], [454, 371, -15], [46
[98, 425, 0], [139, 388, -6], [185, 378, -10], [212, 394, -13], [20
Process finished with exit code 0
```

然后编写数据采集的脚本，首先导入依赖包

```
1. import math
2.
3. import cv2
4. from cvzone.HandTrackingModule import HandDetector
5. import numpy as np
6. import cvzone
7. import pandas as pd
8. import os
9.
10. file_name = 'test.csv'
11.
12.
13. # 设置CSV文件的列名
14. columns = ['real_distance', '5-17', '0-12']
```

5.4编写数据采集脚本

```
16. # 创建空的DataFrame
17. data = pd.DataFrame(columns=columns)
18.
19. # 检查文件是否存在且包含正确的列头
20. if not os.path.isfile(file_name) or os.path.getsize(file_name) == 0:
21.     data.to_csv(file_name, index=False) # 第一次或文件不存在, 写入列头
22. else:
23.     data.to_csv(file_name, mode='a', index=False, header=False) # 追加数据, 不包括列头
24. # 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95, 100, 105, 110, 120, 130, 140, 150, 200
25. real_distance = 20
...
32. while True:
33.     success, img = cap.read()
34.     if success: # 检查帧是否成功读取
35.         # 这个的 draw = False 是指只显示实际距离的检测框, 手部的检测框不显示
36.         hands, img = detector.findHands(img)
37.         if hands:
38.             lmList = hands[0]['lmList']
39.             # hand_type=['Left', 'Right']
40.             # if hand_type[Hand_direction] == hands[0]['type']:
41.             hand_data = []
```

} 获取手掌的所有数据

5.4编写数据采集脚本

```
42. x, y, w, h = hands[0]['bbox']
43. x1, y1, z1 = lmList[5]
44. x2, y2, z2 = lmList[17]
45. x3, y3, z3 = lmList[0]
46. x4, y4, z4 = lmList[12]
47. distance1 = int(math.sqrt((y2 - y1) ** 2 + (x2 - x1) ** 2))
48. distance2 = int(math.sqrt((y4 - y3) ** 2 + (x4 - x3) ** 2))
49. hand_data += [real_distance, distance1, distance2]
50.
51. # 添加当前手部数据到DataFrame
52. data_length = len(data)
53. data.loc[data_length] = hand_data
54. cv2.imshow("Image", img)
55. # 按 'q' 键退出循环
56. if cv2.waitKey(1) & 0xFF == ord('q'):
57.     break
58. cap.release()
59. cv2.destroyAllWindows()
60. # 在捕获循环结束后, 通过指定mode='a' 和header=False来追加数据
61. data.to_csv(file_name, mode='a', index=False, header=False)
```

bbox: 手部检测框的坐标点

lmList: 包含手部关键点的列表。

distance1和distance2: 计算得到的两段关键点间的距离。

将实际距离和计算得到的距离作为数据添加到DataFrame

5.4编写数据采集脚本

将捕获的数据追加到CSV文件中。

随后，我们可以改变real_distance的值，得到在不同距离下手部特征值，部分如右所示：

示例中的图片展示为，在显示25厘米的情况下，四个特征点中，特征5与17、特征0与特征2之间的像素距离

```
test - 副本.csv x
HandDistance > test - 副本.csv
1 real_distance,5_17,0_12
2 25,138,348
3 25,136,358
4 25,138,357
5 25,137,359
6 25,136,358
7 25,137,359
8 25,136,359
9 25,137,361
10 25,137,358
11 25,138,359
12 25,136,357
13 25,136,360
14 25,136,355
15 25,135,356
16 25,135,355
17 25,136,357
18 25,134,357
19 25,136,363
20 25,135,363
21 25,135,363
22 25,135,363
23 25,135,363
```

目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (3学时)

5.1配置开发板的HK驱动

5.2配置本地CVZone环境

5.3测试识别效果

5.4编写数据采集脚本

5.5采集手部特征值与数据格式化

5.6建立测距方程式模型

5.7建立角度方程式模型

5.8模型本地测试

六、模型应用及其部署 (4学时)

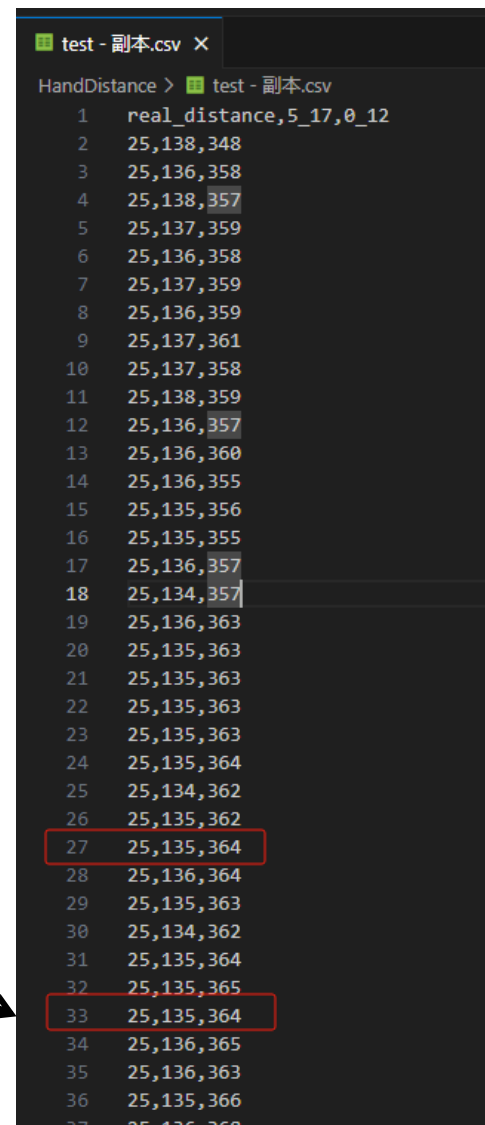
七、运行示例

5.5采集手部特征值与数据格式化

采集之后的数据，如右图所示。

但是不难发现在采集到的数据中，容易出现，两组一模一样的数据，如果出现这种情况，会大大的影响到做回归曲线的，所以需要将相同的组删除掉。处理脚本如下所示：

```
1. import pandas as pd
2. # 定义数据
3. data = pd.read_csv('test - 副本.csv')
4. # 创建DataFrame
5. df = pd.DataFrame(data)
6. # 去重
7. unique_df = df.drop_duplicates(subset=['5_17', '0_12'])
8. # 根据所有列去除重复行
9. unique_df_all = df.drop_duplicates()
10. # 导出数据到CSV
11. unique_df.to_csv('unique_data.csv', index=False)
```



	real_distance,5_17,0_12
1	25,138,348
2	25,136,358
3	25,138,357
4	25,137,359
5	25,136,358
6	25,137,359
7	25,136,359
8	25,137,361
9	25,137,358
10	25,138,359
11	25,136,357
12	25,136,360
13	25,136,355
14	25,135,356
15	25,135,355
16	25,136,357
17	25,134,357
18	25,136,363
19	25,135,363
20	25,135,363
21	25,135,363
22	25,135,363
23	25,135,363
24	25,135,364
25	25,134,362
26	25,135,362
27	25,135,364
28	25,136,364
29	25,135,363
30	25,134,362
31	25,135,364
32	25,135,365
33	25,135,364
34	25,136,365
35	25,136,363
36	25,135,366
37	25,136,368

5.5采集手部特征值与数据格式化

在得到了去重之后的数据，下一步将开始对数据进行回归处理，采用过的方式是Ridge回归模型，L2正则的多项式

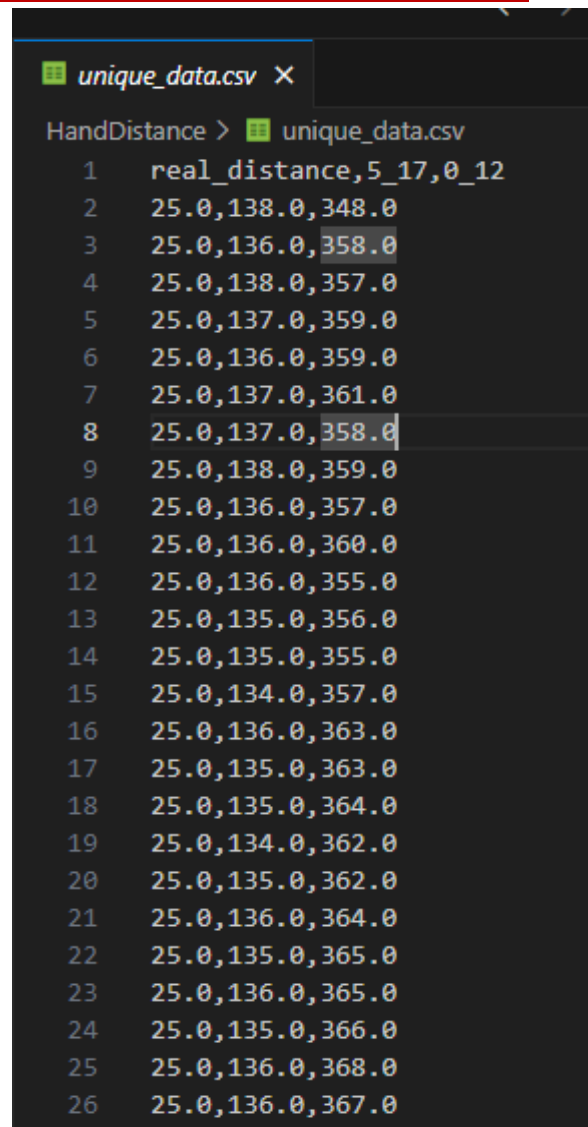
```
17. # 创建多项式特征
18. poly = PolynomialFeatures(degree=2, include_bias=False)
19. X_poly = poly.fit_transform(X)

20. # 分割数据
21. X_train, X_test, y_train_log, y_test_log = train_test_split(X_poly, y_log, test_size=0.2,
    random_state=42)
22. # 使用Ridge回归模型
23. model = Ridge(alpha=1.0)
24. model.fit(X_train, y_train_log)

25. # 预测
26. y_log_pred = model.predict(X_test)
27. y_pred = np.exp(y_log_pred)
28. r_square_log = model.score(X_test, y_test_log)

30. # 输出模型的参数
31. coef = model.coef_
32. intercept = model.intercept_

35. # 创建一个管道，包含多项式特征和Ridge回归模型 L2正则
36. pipeline = make_pipeline(PolynomialFeatures(degree=2, include_bias=False), Ridge(alpha=1.0))
37. pipeline.fit(X_train, y_train_log)
38. # 获取多项式特征的名称
39. features = poly.get_feature_names_out(['x1', 'x2'])
```



	real_distance	5_17	0_12
1	25.0	138.0	348.0
2	25.0	136.0	358.0
3	25.0	138.0	357.0
4	25.0	137.0	359.0
5	25.0	136.0	359.0
6	25.0	137.0	361.0
7	25.0	137.0	358.0
8	25.0	138.0	359.0
9	25.0	136.0	357.0
10	25.0	136.0	360.0
11	25.0	136.0	355.0
12	25.0	135.0	356.0
13	25.0	135.0	355.0
14	25.0	134.0	357.0
15	25.0	136.0	363.0
16	25.0	135.0	363.0
17	25.0	135.0	364.0
18	25.0	134.0	362.0
19	25.0	135.0	362.0
20	25.0	136.0	364.0
21	25.0	135.0	365.0
22	25.0	136.0	365.0
23	25.0	135.0	366.0
24	25.0	136.0	368.0
25	25.0	136.0	367.0

5.5采集手部特征值与数据格式化

得到回归方程之后，将结果写入json文件中，具体实现如下所示：

```
59. import json
60. # 构建模型数据和方程式的字典
61. model_data = {
62.     "equation": equation,
63.     "description": "这是使用具有多项式特征的岭回归预测基于 x1 和 x2 的实际距离的模型方程。",
64.     "r_square_log": r_square_log, # 存储R^2分数
65.     "model_parameters": {
66.         "coefficients": coef.tolist(), # 将Numpy数组转换为列表
67.         "intercept": intercept
68.     },
69.     "features": features.tolist() # 确保特征名称以适当的格式存储
70. }
71.
72. # 写入模型数据到JSON文件
73. with open('ridge_regression_model.json', 'w') as file:
74.     json.dump(model_data, file, indent=4)
75.
76. print("模型方程式和参数已成功保存到JSON文件。")
```

5.5采集手部特征值与数据格式化

我们可以看到这段代码实现了一个典型的机器学习流程，从数据预处理到模型训练、评估、解释和保存。代码中使用的岭回归和多项式特征是解决回归问题中的常用技术。

```
{ } ridge_regression_model.json X
HandDistance > { } ridge_regression_model.json > ...
1 {
2   "equation": "y = e^(- 0.004*x1 - 0.007*x2 - 0.000*x1^2 + 0.000*x1 x2 - 0.000*x2^2 + 5.514)",
3   "description": "\u8fd9\u662f\u4f7f\u7528\u5177\u6709\u591a\u9879\u5f0f\u7279\u5f81\u7684\u56de\u5f52\u9884\u6d4b\u57f7",
4   "r_square_log": 0.994877034295452,
5   "model_parameters": {
6     "coefficients": [
7       -0.004067305655731796,
8       -0.006693224559602325,
9       -0.00012876435160600512,
10      9.590659661135591e-05,
11      -1.1131910088514775e-05
12    ],
13    "intercept": 5.5139729579962085
14  },
15   "features": [
16     "x1",
17     "x2",
18     "x1^2",
19     "x1 x2",
20     "x2^2"
21   ]
22 }
```

上图是最终的到的方程式，保存在了json文件中

目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (3学时)

5.1配置开发板的HK驱动

5.2配置本地CVZone环境

5.3测试识别效果

5.4编写数据采集脚本

5.5采集手部特征值与数据格式化

5.6建立测距方程式模型

5.7建立角度方程式模型

5.8模型本地测试

六、模型应用及其部署 (4学时)

七、运行示例

5.6建立测距方程式模型

得到了方程式之后，我们可以通过代码的方式，通过这个方程，来计算出距离了，实现方式如下：

```
1. import numpy as np
2. import json
3. import math
4.
5. def predict_distance(x1, x2, x3, x4, y1, y2, y3, y4) -> float:
6.     # with open('ridge_regression_model.json', 'r') as file:
7.     #     model_data = json.load(file)
8.     model_data = {
9.         "equation": "y = e^( - 0.004*x1 - 0.007*x2 - 0.000*x1^2 + 0.000*x1 x2 - 0.000*x2^2 + 5.514)",
10.        "description":
11.        "\u8fd9\u662f\u4f7f\u7528\u5177\u6709\u591a\u9879\u5f0f\u7279\u5f81\u7684\u5cad\u56de\u5f52\u9884\u6d4b\u57fa\u4e8e x1 \u54c x2 \u7684\u5b9e\u9645\u8ddd\u79bb\u7684\u6a21\u578b\u65b9\u7a0b\u3002",
12.        "r_square_log": 0.994877034295452,
13.        "model_parameters": {
14.            "coefficients": [
15.                -0.004067305655731796,
16.                -0.006693224559602325,
17.                -0.00012876435160600512,
18.                9.590659661135591e-05,
19.                -1.1131910088514775e-05
20.            ],
21.            "intercept": 5.5139729579962085
22.        },
23.    }
```

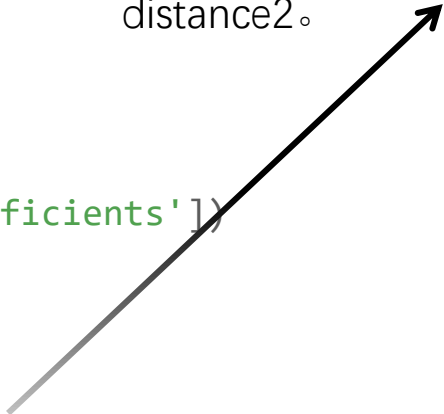
函数定义：

predict_distance(x1, x2, x3, x4, y1, y2, y3, y4)：函数接受八个参数，每两个参数一组，代表手部关键点的 x 和 y 坐标。

5.6建立测距方程式模型

```
22.     "features": [  
23.         "x1",  
24.         "x2",  
25.         "x1^2",  
26.         "x1 x2",  
27.         "x2^2"  
28.     ]  
29. }  
30. coefficients = np.array(model_data['model_parameters']['coefficients'])  
31. intercept = model_data['model_parameters']['intercept']  
32.  
33. """根据多项式特征计算预测距离"""  
34. distance1 = int(math.sqrt((y2 - y1) ** 2 + (x2 - x1) ** 2))  
35. distance2 = int(math.sqrt((y4 - y3) ** 2 + (x4 - x3) ** 2))  
36. features = [distance1, distance2, distance1**2, distance1*distance2, distance2**2]  
37. features = np.array(features) # 确保特征是numpy数组  
38. y_log_predicted = np.dot(features, coefficients) + intercept  
39. distance_predicted = np.exp(y_log_predicted) # 取指数获得原始距离  
40. return distance_predicted
```

计算手部关键点间得到距离：
使用 `math.sqrt` 和距离公式计算由关键点坐标确定的两段距离 `distance1` 和 `distance2`。



目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (3学时)

5.1配置开发板的HK驱动

5.2配置本地CVZone环境

5.3测试识别效果

5.4编写数据采集脚本

5.5采集手部特征值与数据格式化

5.6建立测距方程式模型

5.7建立角度方程式模型

5.8模型本地测试

六、模型应用及其部署 (4学时)

七、运行示例

5.7建立角度方程式模型

角度的方程更加简单，只需要得到像素，与手部识别框的位置关系即可，得到每个像素得到的角度值。

代码如下所示：

```
1. def predict_angle(x_val, y_val):
2.     # 广角摄像头一半的角度
3.     camera_anlge = 60 / 2
4.     # 图像中心的坐标
5.     center = [320, 240]
6.     # 手部矩形框与中心点的坐标差值, ( $\Delta x$ ,  $\Delta y$ )
7.     center_df = [center[0]-x_val, center[1]-y_val]
8.     # print(center_df)
9.     # 矩形框中心点坐标差 $\Delta x$ 乘以每一个像素对应的广角摄像头的角度值
10.    angle_x = -center_df[0] * (camera_anlge / center[0])
11.    # angle_x = center_df[0] * (camera_anlge / center[0])
12.    # 同理可得垂直方向的
13.    angle_y = center_df[1] * (camera_anlge / center[1])
14.
15.    return [angle_x, angle_y]
```

目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (3学时)

5.1配置开发板的HK驱动

5.2配置本地CVZone环境

5.3测试识别效果

5.4编写数据采集脚本

5.5采集手部特征值与数据格式化

5.6建立测距方程式模型

5.7建立角度方程式模型

5.8模型本地测试

六、模型应用及其部署 (4学时)

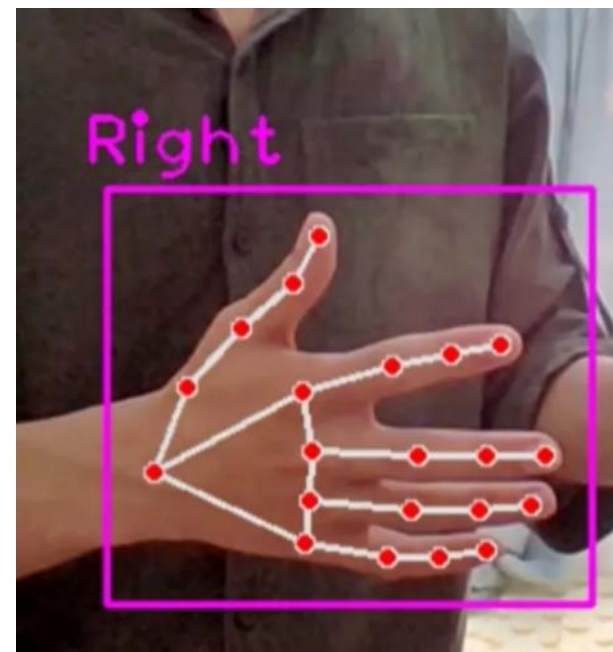
七、运行示例

5.8 模型本地测试

根据测试信息可以得到距离与角度。（这里测试的值与图片所反馈的不符，在实际中我们需要自行测试，直到得到准确的值）

```
(base) root@davinci-mini:/opt/Dog/UDPControl# python HandDistance/main.py
Find 1 devices!

u3v device: [0]
device model name: MV-CS060-10UC-PRO
user serial number: DA0275184
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
距离: 76.08354297534035
角度 -3.375
距离: 67.79175314160902
角度 -1.78125
距离: 67.53255065873913
角度 -2.34375
距离: 69.22358033074413
角度 -1.6875
距离: 69.52555997559446
角度 -1.6875
距离: 69.52555997559446
角度 -1.59375
距离: 69.82730233095928
角度 -1.96875
```



目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (3学时)
- 六、模型应用及其部署 (4学时)
 - 6.1 手部关键点检测
 - 6.2 部署流程
 - 6.3 导入必备的包
 - 6.4 建立通信
 - 6.5 发送心跳包
 - 6.6初始化手部检测器
 - 6.7 建立两个标志位
 - 6.8 摄像头测距
 - 6.9 机器狗执行动作
- 七、运行示例

六、模型应用及其部署

整体流程

手部关键点检测

导入必备包

建立通信

发送心跳包

初始化检测器和变量

测距

传递距离给机器狗

目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (3学时)

六、模型应用及其部署 (4学时)

6.1 手部关键点检测

6.2 部署流程

6.3 导入必备的包

6.4 建立通信

6.5 发送心跳包

6.6 速度函数

6.7 初始化手部检测器

6.8 建立两个标志位

6.9 摄像头测距

6.10 机器狗执行动作

七、运行示例

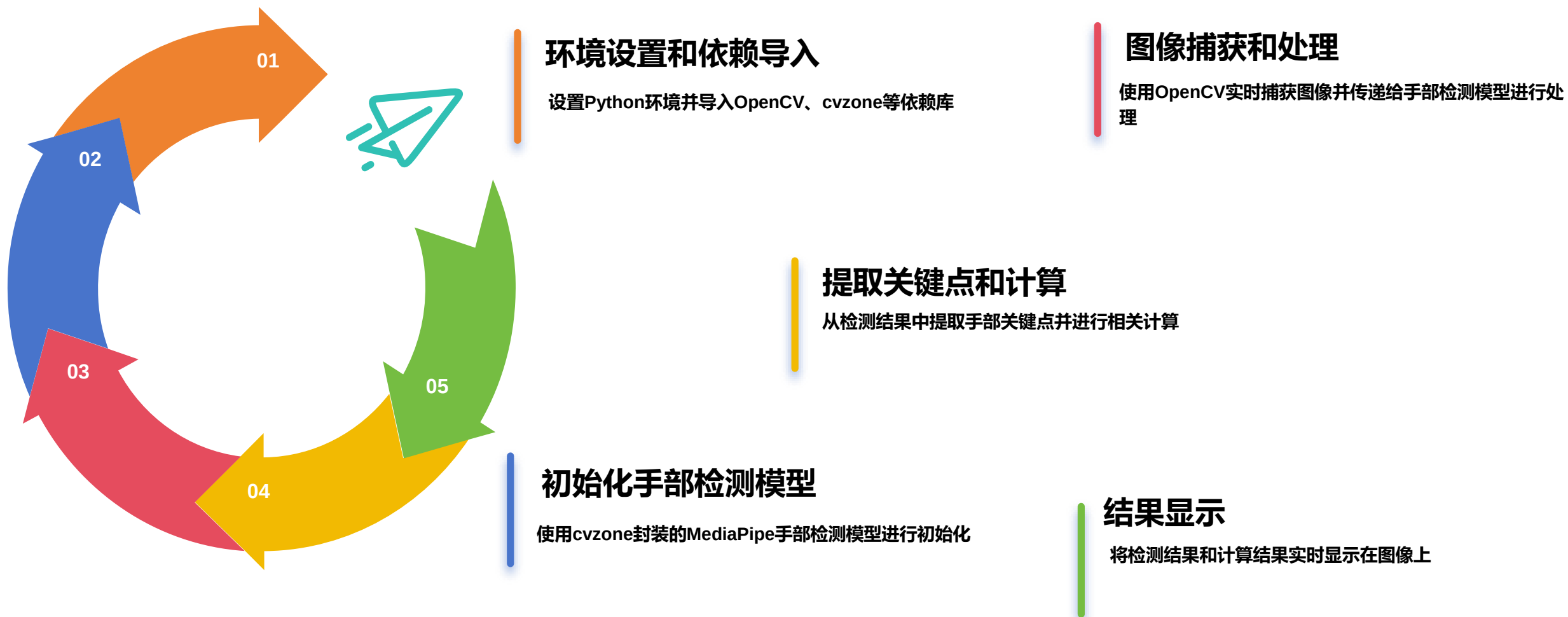
6.1 手部关键点检测

- 1.功能：手部关键点检测模型可以在输入图像中找到手的21个关键点，并返回这些点的坐标。
- 2.使用库：cvzone（封装了MediaPipe的功能）
- 3.检测流程：
 - a.初始化检测器：通过HandDetector类初始化手部检测器。
 - b.检测手部：使用findHands方法在每一帧图像中检测手部，并返回检测到的手部信息和处理后的图像。
 - c.提取关键点：从检测结果中提取21个关键点的坐标，用于进一步处理。

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (3学时)
- 六、模型应用及其部署 (4学时)
 - 6.1 手部关键点检测
 - 6.2 部署流程
 - 6.3 导入必备的包
 - 6.4 建立通信
 - 6.5 发送心跳包
 - 6.6 速度函数
 - 6.7 初始化手部检测器
 - 6.8 建立两个标志位
 - 6.9 摄像头测距
 - 6.10 机器狗执行动作
- 七、运行示例

6.2 部署流程



目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (3学时)
- 六、模型应用及其部署 (4学时)
 - 6.1 手部关键点检测
 - 6.2 部署流程
 - 6.3 导入必备的包
 - 6.4 建立通信
 - 6.5 发送心跳包
 - 6.6 速度函数
 - 6.7 初始化手部检测器
 - 6.8 建立两个标志位
 - 6.9 摄像头测距
 - 6.10 机器狗执行动作
- 七、运行示例

6.3导入必备的包

```
1. #!/usr/bin/env python
2. # -*- coding: utf-8 -*-
3.
4. import cv2 # 图片处理三方库, 用于对图片进行前后处理
5. import logging
6. from typing import *
7. import sys
8. sys.path.append('camera/')
9. from HKcamera import getImage
10. from det_utils import get_labels_from_txt, letterbox, scale_coords, nms, draw_bbox # 模型
    前后处理相关函数
11.
12. # 导入定义的机器狗相关的包
13. from threading_utils.ThreadTemplates import * # 线程的模板与基本轴指令的发送
14. from sendcommand import SendToCommand, heartbeat # 发送简单指令与心跳包
15. from socketnetwork import network_utils # 通信函数
16. from command.udp_command import * # 存放各种结构体与状态数据、指令
17. from speeds.sportspeed import * # 运动精准控制的基础版模型
18. from robotstatuswatcher.listener import * # 监听机器狗状态
```

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (3学时)
- 六、模型应用及其部署 (4学时)
 - 6.1 手部关键点检测
 - 6.2 部署流程
 - 6.3 导入必备的包
 - 6.4 建立通信
 - 6.5 发送心跳包
 - 6.6 速度函数
 - 6.7 初始化手部检测器
 - 6.8 建立两个标志位
 - 6.9 摄像头测距
 - 6.10 机器狗执行动作
- 七、运行示例

6.4 建立通信函数

简单控制第一步：建立通信链接，socketnetwork/network_utils.py文件定义了一个函数 `setup_socket_and_address`，用于创建一个UDP套接字，并设置机器狗的IP地址和端口号作为目标地址。代码如下：

```
1. import socket
2. from typing import *
3.
4. def setup_socket_and_address(dest_ip = '192.168.1.120',
    port=43893) -> Tuple[socket.socket, Tuple[str, int]]:
5.     # 创建UDP套接字
6.     sfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
7.
8.     # 设置目标地址
9.     target_address = (dest_ip, port)
10.    # print(target_address)
11.
12.    return sfd, target_address
```

它接受两个参数：dest_ip 和 port，这两个参数分别指定了目标IP地址和端口号

创建一个新的UDP套接字

将传入的IP地址和端口号组合成一个元组，这个元组将用作套接字的目标地址。

目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (3学时)

六、模型应用及其部署 (4学时)

6.1 手部关键点检测

6.2 部署流程

6.3 导入必备的包

6.4 建立通信

6.5 发送心跳包

6.6 速度函数

6.7 初始化手部检测器

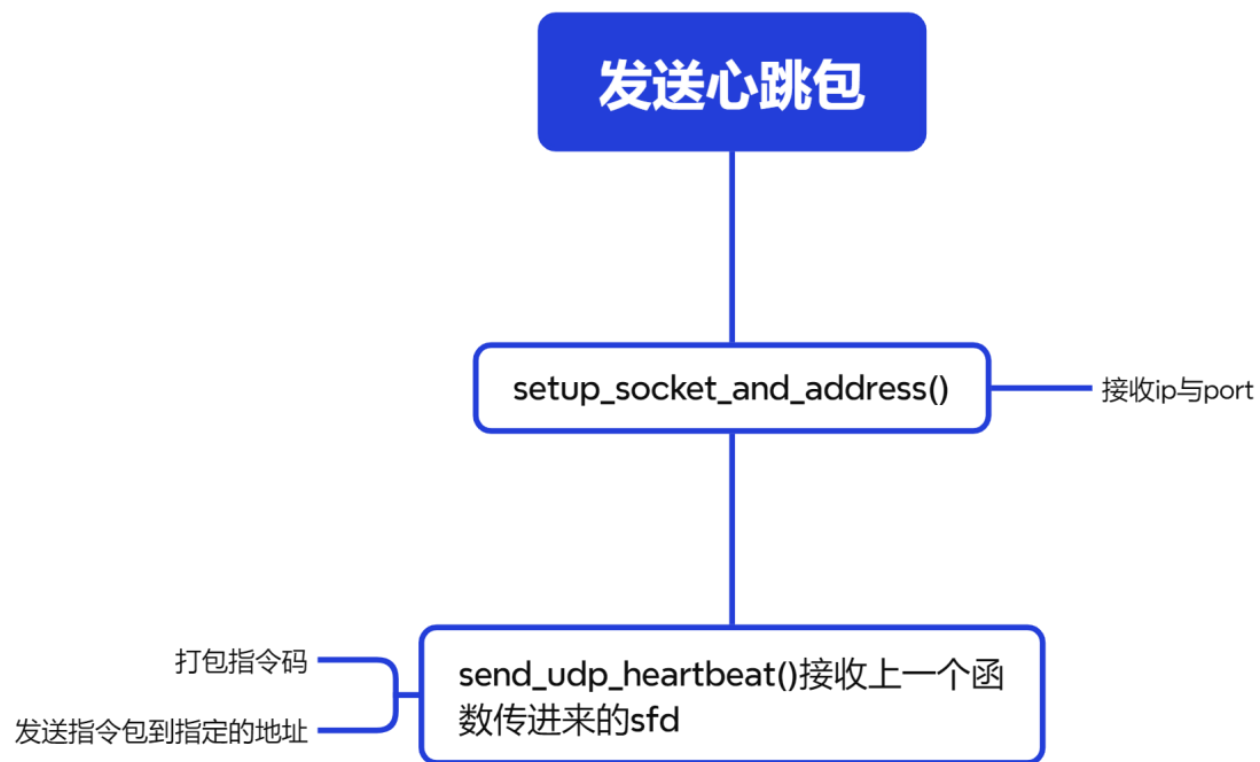
6.8 建立两个标志位

6.9 摄像头测距

6.10 机器狗执行动作

七、运行示例

6.5 发送心跳包



6.5 发送心跳包

心跳包是一种周期性发送的信号，用于确保网络连接的活跃状态，防止因长时间无通信而被网络设备断开连接。发送心跳包代码位于sendcommand/heartbeat.py，代码如下：

```
4. import socket
5. import struct
6. import time
7.
8. def send_udp_heartbeat(sfd, target_address, code=0x21040001, parameters_size=0, type=0, heartbeat_interval=0.25) -> None:
9.     # 打包心跳指令数据
10.     heartbeat_command = struct.pack('<III', code, parameters_size, type)
11.
12.     try:
13.         while True:
14.             # 记录发送前的时间
15.             start_time = time.time()
16.
17.             # 发送心跳包
18.             sfd.sendto(heartbeat_command, target_address)
19.
20.             # 发送后的延时
21.             time.sleep(max(0, heartbeat_interval - (time.time() - start_time)))
22.
23.     except KeyboardInterrupt:
24.         print("Heartbeat sending stopped by user.")
25.     finally:
26.         # 关闭socket
27.         sfd.close()
```

打包心跳包

将打包好的心跳包发送到指定的地址

6.5 发送心跳包

定义好了之后，可以在别的文件中启动心跳线程，代码如下：

```
1. # 建立通讯
2. sfd, target_address = network_utils.setup_socket_and_address()
3.
4. # 定义发送心跳包的线程执行体
5. # 注意*args必须以tuple形式传入，包括sfd和target_address
6. heartbeat_thread = MyRepeatThread("HeartbeatThread", heartbeat.send_udp_heartbeat, 0.25, None, sfd, target_address)
7. heartbeat_thread.start()
```

heartbeat_thread.start() 启动了心跳线程，它将按照设定的时间间隔（这里是0.25秒）发送心跳包。

目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (3学时)

六、模型应用及其部署 (4学时)

6.1 手部关键点检测

6.2 部署流程

6.3 导入必备的包

6.4 建立通信

6.5 发送心跳包

6.6 速度函数

6.7 初始化手部检测器

6.8 建立两个标志位

6.9 摄像头测距

6.10 机器狗执行动作

七、运行示例

6.6 速度函数

go_straight这个函数用于控制设备直线行驶。long: 表示要行驶的距离，单位是米。speedgear: 表示设备的档位，影响速度，其默认值为3。

times: 表示行驶时间，单位是秒。

函数会根据时间和速度计算行驶的距离。如果long不是9999，函数会根据距离和速度计算所需的时间。最终，函数返回行驶所需的时间（或距离）和速度值。

```
1. def go_straight(long, speedgear=3, times = None, obs_void_distance = None):
2.     # 档位速度只是参考值, 具体速度按照实际来判断, 默认速度是3档
3.     speedgear_ditc = {
4.         10.     }
5.
6.     val = speedgear_ditc[speedgear]
7.     # 如果传入的是时间, 计算出已经走过的距离, Long传入9999是为在特殊情况才会有下面的情况
8.     if times != None and obs_void_distance != None and long == 9999:
9.         speed_per_second_meter = (-7.237e-09) * val ** 2 + 0.0002933 * val - 1.66
10.        # 这里要计算出避障的距离的时间消耗
11.        times_obs_void_distance = obs_void_distance / speed_per_second_meter
12.
13.        long = abs((times - times_obs_void_distance) * speed_per_second_meter)
14.    return long
15.
16. else:
17.     if long < 0:
18.         val = -val
19.         speed_per_second_meter = (-1.5059e-08) * val ** 2 - (4.1944e-04) * val - 2.1804
20.         times = abs(long / speed_per_second_meter)
21.     else:
22.         speed_per_second_meter = (-7.237e-09) * val ** 2 + 0.0002933 * val - 1.66
23.         times = long / speed_per_second_meter
24. return [times, val]
```

根据时间算出距离

根据距离算出时间

6.6 速度函数

`translate_left_and_right(long, speedgear=3)`: 这个函数用于控制设备左右平移。参数和`go_straight`函数类似，但速度值是不同的字典`speedgear_ditc`。根据`long`的正负，选择相应的速度计算公式来计算时间。返回值是平移所需的时间和速度值。

```
1. def translate_left_and_right(long, speedgear=3):
2.     # 档位速度只是参考值, 具体速度按照实际来判断, 默认速度是3档
3.     speedgear_ditc = {
4.         1: 14000,
5.         2: 18000,
6.         3: 21000,
7.         4: 24000,
8.         5: 27000,
9.         6: 34000,
10.    }
11.    val = speedgear_ditc[speedgear]
12.    if long < 0:
13.        val = -val
14.        speed_per_second_meter = -(1.6e-05) * val - 0.2256
15.        times = abs(long / speed_per_second_meter)
16.    else:
17.        speed_per_second_meter = (1.517e-05) * val - 0.1748
18.        times = long / speed_per_second_meter
19.    return [times, val]
```

相应的速度计算公式

6.6 速度函数

revolve_left_and_right(angle) :

这个函数用于控制设备左右旋转。**angle:** 表示要旋转的角度，单位是度。

get_stack_info方法，用于根据调用的文件名和角度的大小，函数会设置不同的时间和速度值。

最终，函数返回旋转所需的时间和速度值。

```
1. def revolve_left_and_right(angle):
2.     def find_closest_output(input_value):
3.         # 判断角度是否超过了360度, 超过了则需要则算掉
4.         if input_value > 360:
5.             input_value = input_value / (input_value % 360)
6.
7.         # 输出到输入的映射字典
8.         output_to_input_map = {
9.             .....
35.         }
39.         # 获取所有输出键并将其转换为列表
40.         keys = list(output_to_input_map.keys())
41.         # 寻找与输入值最接近的输出键
42.         closest_key = min(keys, key=lambda x: abs(x - input_value))
43.         # 返回与最接近键关联的输入值
44.         return output_to_input_map[closest_key]
45.
46.     caller_function = StackTraceParser()
47.     # print(caller_function.get_formatted_stack_info())
48.
49.     # 根据栈的追踪, 如果旋转的调用执行不同的旋转方式
50.     if caller_function.get_stack_info()[0]['file'].split('/')[1] == 'hand_distance_main.py':
65.         .....
66.     else:
67.         times = find_closest_output(abs(angle)) # 获取时间与角度的关系
68.         if angle < 0:
69.             val = -10000
70.         else:
71.             val = 10000
72.
73.     return [times, val]
```

目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (3学时)

六、模型应用及其部署 (4学时)

6.1 手部关键点检测

6.2 部署流程

6.3 导入必备的包

6.4 建立通信

6.5 发送心跳包

6.6 速度函数

6.7 初始化手部检测器

6.8 建立两个标志位

6.9 摄像头测距

6.10 机器狗执行动作

七、运行示例

6.7 初始化手部检测器

实例化手部检测器，通过设置特定的置信度阈值和最大检测数量，创建了一个手部检测器实例，该实例能够用于后续的手部检测任务

- **detectionCon**: 这是一个参数，用于设置检测置信度的阈值。在这个例子中，阈值被设置为0.8，意味着只有当检测算法的置信度达到或超过80%时，才会认为检测到的手部是有效的。这是一个质量控制的措施，用于过滤掉那些置信度较低的检测结果，提高检测的准确性。
- **maxHands**: 这个参数设置了可以检测到手部的最大数量。在这个例子中，数量被设置为2，表示该检测器会尝试在图像中找到最多两只手。

1. # 实例化手部检测器

2. `detector = HandDetector(detectionCon=0.8, maxHands=2)`

目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (3学时)

六、模型应用及其部署 (4学时)

6.1 手部关键点检测

6.2 部署流程

6.3 导入必备的包

6.4 建立通信

6.5 发送心跳包

6.6 速度函数

6.7 初始化手部检测器

6.8 建立两个标志位

6.9 摄像头测距

6.10 机器狗执行动作

七、运行示例

6.8 建立两个标志位

1. # 标志位
2. HKflag = 1
3. Dogflag = 0

Hkflag的意义是进入到摄像头的循环中，表示还在测量距离，加入测量到了，那么回零，跳出摄像头，Dogflag变为1，进入到狗做动作的循环中去。

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (3学时)
- 六、模型应用及其部署 (4学时)
 - 6.1 手部关键点检测
 - 6.2 部署流程
 - 6.3 导入必备的包
 - 6.4 建立通信
 - 6.5 发送心跳包
 - 6.6 速度函数
 - 6.7 初始化手部检测器
 - 6.8 建立两个标志位
 - 6.9 摄像头测距
 - 6.10 机器狗执行动作
- 七、运行示例

6.9 摄像头测距

这段代码通过摄像头检测手部，计算其角度和距离，并根据距离阈值控制标志位的状态。

```
1. # 无限循环，除非在循环内部被显式地中断
2. while True:
3.     # 只要 HKflag 标志为真，就继续循环
4.     while HKflag:
5.         # 获取图像，getImage() 函数需要自行定义或导入
6.         img = getImage()
7.         # 使用 detector 的 findHands 方法检测图像中的手部
8.         # draw=False 表示不显示手部的检测框，只关注实际距离的检测框
9.         hands, img = detector.findHands(img, draw=False)
10.        # 初始化一个空列表，用于存储某些数据
11.        list0 = []
12.        # 如果检测到至少一只手
13.        if hands:
14.            # 获取第一只手的标记点列表
15.            lmList = hands[0]['lmList']
16.        .....
30.        # 如果距离超过10厘米，更新 HKflag 和 Dogflag 标志
31.        # 这里的条件被注释掉了，因此实际上无论角度如何，只要距离超过10厘米就会更新标志
32.        if abs(distanceCM) > 10:
33.            HKflag = 0 # 更新 HKflag 标志
34.            Dogflag = 1 # 更新 Dogflag 标志
40.            break
```

目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (3学时)

六、模型应用及其部署 (4学时)

6.1 手部关键点检测

6.2 部署流程

6.3 导入必备的包

6.4 建立通信

6.5 发送心跳包

6.6 速度函数

6.7 初始化手部检测器

6.8 建立两个标志位

6.9 摄像头测距

6.10 机器狗执行动作

七、运行示例

6.10 机器狗执行动作

这段代码控制机器狗执行动作，通过循环和多线程处理动作命令。while Dogflag:循环在Dogflag为真时执行，进入循环后记录日志，表明进入动作执行阶段。

```
1. # 只要 Dogflag 标志为真，就持续循环
2. while Dogflag:
3.     # 记录进入循环的信息到日志中
4.     logging.info('-----进入狗循环中-----')
5.     # 定义动作ID与对应参数的映射字典
6.     actions_params = {
7.         1: [(distanceCM/100,)], # 动作ID为1，参数是距离（以米为单位），使用默认速度
8.         2: [(angle,)] # 动作ID为2，参数是角度
9.     }
10.    # 定义动作顺序列表，包含动作ID和参数列表索引
11.    # 动作将按照此顺序执行，例如先执行动作ID为2的转弯动作，再执行动作ID为1的前进动作
12.    actions_sequence = [(2, 0), (1, 0)]
13.    # 遍历动作顺序列表，执行每个动作
14.    for action_id, params_index in actions_sequence:
15.        .....
32.        else:
33.            # 如果参数不是元组或列表，记录错误日志
34.            logging.error('params 必须是一个 tuple 或者 列表')
35.    # 更新 HKflag 和 Dogflag 标志的状态
36.    HKflag = 1 # 将 HKflag 设置为真
37.    Dogflag = 0 # 将 Dogflag 设置为假，从而退出循环
```

目录

- 一、实验前置准备
- 二、实验过程描述
- 三、实验环境准备
- 四、工程文件目录
- 五、数据工程
- 六、模型应用及其部署
- 七、运行示例

七、运行示例

首先在文件夹的目录下执行先执行这个指令

```
. /usr/local/Ascend/mxVision-5.0.RC3/set_env.sh && . /usr/local/Ascend/ascend-toolkit/set_env.sh
```

然后再输入

```
python hand_distance_main.py
```

在对机器狗做出相对应的手势即可。

```
(base) root@davinci-mini:/opt/lab/HandTracking# python hand_distance_main.py
Find 1 devices!

u3v device: [0]
device model name: MV-CS060-10UC-PRO
user serial number: DA0275184
成功连接到ip: 192.168.1.100, port: 43897
Starting HeartbeatThread
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
INFO:root:-----进入狗循环中-----
INFO:root:-----
INFO:root:action_func: <function action_revolve_left_and_right at 0xe7ffec042040>
INFO:root:params: (-3.5625,)
Starting ACTION_revolve_left_and_right
INFO:root:ACTION_revolve_left_and_right由于超过时间阈值0秒，系统自动停止！
INFO:root:离开线程: ACTION_revolve_left_and_right
INFO:root:-----
INFO:root:action_func: <function action_go_straight at 0xe7ffec031e50>
INFO:root:params: (0.4200209919221258,)
Starting ACTION_pan_back_and_forth_thread
INFO:root:ACTION_pan_back_and_forth_thread由于超过时间阈值1.7483973489049192秒，系统自动停止！
INFO:root:离开线程: ACTION_pan_back_and_forth_thread
```

感谢

版权所有©2024，华为技术有限公司，保留所有权利。

本资料是华为的保密信息，所有内容仅供华为授权的培训客户内部使用，禁止用于任何其他用途。未经许可，任何人不得对本资料进行复制、修改、改编、也不得将本资料或其任何部分或基于本资料的衍生作品提供给他人。

华为开发者创新中心
<https://connect.huaweicloud.com>